George Grätzer

Exercise Help and Solutions

Vibe Coding for Legal Professionals

Hints in Part I; complete solutions in Part II

October 24, 2026
Release `legal-copyedit-2026-10-24.2`

Contents

How to Use This Companion iv

I Hints 1

1 Build a Matter Timeline 2

2 Check the Matter Records 3

3 From Eighteen Records to 12,000 Events 4

4 Check Consultation Completion 5

5 Catalogue Matter Documents 6

6 Check a Document Register 7

7 Prepare a Limited Management Extract 8

8 Match Requests and Responses 10

9 Define the Matters in a Report 11

10 Report Monthly Turnaround 12

11 Monitor Weekly Requests 14

12 Build a Production Index 15

13 Prepare Administrative Letters 16

14 Track Professionally Supplied Dates 18

15 Search Names and Parties 19

16 Organize Legal Authorities 21

17 Compare Document Versions 22

18 Check Time Entries and Totals 23

19 Review Files Recorded as Closed 24

20 Test the Applications 26

21 Package a Reproducible Demonstration 27

22 Check Application-Review Records 29

23 Build an Integrated Matter Worklist 30

II Solutions 32

1 Build a Matter Timeline 33

2 Check the Matter Records 34

3 From Eighteen Records to 12,000 Events 36

4 Check Consultation Completion 37

5 Catalogue Matter Documents 39

6 Check a Document Register 41

7 Prepare a Limited Management Extract 42

8 Match Requests and Responses 45

9 Define the Matters in a Report 47

10 Report Monthly Turnaround 49

11 Monitor Weekly Requests 51

12 Build a Production Index 53

13 Prepare Administrative Letters 54

14 Track Professionally Supplied Dates 56

15 Search Names and Parties 59

16 Organize Legal Authorities 61

17 Compare Document Versions **63**

18 Check Time Entries and Totals **65**

19 Review Files Recorded as Closed **67**

20 Test the Applications **69**

21 Package a Reproducible Demonstration **71**

22 Check Application-Review Records **73**

23 Build an Integrated Matter Worklist **76**

How to Use This Companion

The chapter and exercise numbers match the main book. Each chapter has five exercises. Try the question first; consult its hint in Part I only when needed. Part II repeats the question and gives a full answer.

All numerical answers concern the stated synthetic inputs and rules. They are not professional conclusions about actual clients, deadlines, conflicts, or fees. A different layout or harmless ordering can be acceptable; different record identities or classifications need explanation. Preserve your own input, output, and comparison rather than treating a printed expected answer as evidence that you ran a program.

Each chapter's controlled change is described in the main book, and its saved run is in `reference_outputs/experiments`; the exercises go beyond it. Many ask you to predict before you run. Verify a prediction by running the supplied program on a copy of the chapter folder, and compare identities, not only counts. Some design exercises admit several good implementations; their observable behaviour matters more than matching one piece of code. When an exercise asks for a command or a program change, you may ask an assistant to carry it out on a copy and return the unchanged inputs, the changed program, and the complete run record. You still need to predict the result and inspect the evidence yourself.

Part I

Hints

Chapter 1

Build a Matter Timeline

Exercise 1.1

List the event identifiers for M02. Why must they stay out of the display when you select M01?

Hint. Look at the matter_id column, not the client_id column.

Exercise 1.2

Copy the file and replace E03's summary with `A & B <b>review</b>`. What exactly should the page display? Explain why the angle brackets must remain visible and the word `review` must not become bold.

Hint. Think about what a browser does with angle brackets in a page's source.

Exercise 1.3

Design a test to catch a timeline that selects by client instead of matter. State the matter to select, the identifiers you expect, and one identifier that must not appear.

Hint. Choose a client with two matters.

Exercise 1.4

Open your run's `accepted.csv`. Which column points back to the input record? Find its value for E05 and explain why it is not 5.

Hint. Count the header line.

Exercise 1.5

Change one letter in E03's summary in a copy of the input, then run again. Which lines in the run record change? Which stay the same? What does this show about fingerprints?

Hint. Compare `input_sha256` with the other two fingerprints.

Chapter 2

Check the Matter Records

Exercise 2.1

Copy the problem file. Restore E14's matter to M05 and E17's client to C03. Change nothing else. Predict the accepted count, held count, held identifiers, and number of reasons.

Hint. Two of the six edits are undone; one of the remaining four affects two rows.

Exercise 2.2

Design a test to distinguish matching by client from matching by matter and request. State the input edit and the record that should be held.

Hint. Move a response to the client's other matter, keeping its request identifier.

Exercise 2.3

These rules hold an otherwise valid event whose summary is blank; another office might accept it. Does that make this checker wrong? What would need to change?

Hint. Distinguish an implementation error from a rule choice.

Exercise 2.4

Copy the problem file and leave its existing problems in place. Remove SRC01 from E01. How many records should be held? What reasons should E01 and E02 have?

Hint. E02 depends on E01.

Exercise 2.5

An assistant reports "12 accepted, 6 held" on the unchanged problem file, and you cannot see its program. Which review row would you inspect first, and what would you expect to find?

Hint. Six edits, seven held records: which record is the odd one?

Chapter 3

From Eighteen Records to 12,000 Events

Exercise 3.1

Count the events for client C0001 and for matter M0001. List the identifiers in each group.

Hint. Each client has three matters of four events each.

Exercise 3.2

Why does a total of 12,000 events fail to show that every event belongs to the right matter?

Hint. A total can be right while identities are wrong.

Exercise 3.3

Run the irregular file. Identify the three held records and the reason for each.

Hint. Look at I03's events.

Exercise 3.4

Run the irregular file with `--matter I04`, then with `--matter I05`. Describe both results. Why must they differ?

Hint. One matter is registered, one is not.

Exercise 3.5

Copy the large file and change E00010's identifier to E00002. Predict the accepted and held counts for the whole file and the selected counts for M0001. Which records should be held? Why should the selected report change?

Hint. Duplicates are held in pairs, and responses depend on their requests.

Chapter 4

Check Consultation Completion

Exercise 4.1

Consider cancelling one appointment at a time. Which cancellations would raise the percentage, and to what? Which would lower it? Explain your predictions, then run one case to check.

Hint. Cancelling removes an appointment from the denominator. Ask whether it was also in the numerator.

Exercise 4.2

What percentage should appear if every appointment is cancelled? Explain your answer.

Hint. What is the denominator?

Exercise 4.3

Set A05's completion date to 2026-09-04, one day before its scheduled date. Predict its ledger entry and the resulting percentage.

Hint. A completion cannot precede the appointment.

Exercise 4.4

Copy A08's complete row and append the copy as a ninth appointment. In the copy, change only its status to `completed` and its completion date to `2026-09-08`; keep the appointment ID A08 and all other values. What should happen to the headline percentage? Why is withholding it better than calculating it?

Hint. Duplicate identifiers are a structural problem.

Exercise 4.5

Before running with `--as-of 2026-09-05`, predict the numerator and denominator. Which appointments leave the denominator, and why? Run the program to check.

Hint. Only appointments scheduled on or before the reporting date are eligible.

Chapter 5

Catalogue Matter Documents

Exercise 5.1

The same input gives totals of eight, five, and seven. Explain each total, naming the records that account for the differences.

Hint. Count rows, matter–document pairs, and matter–document–version triples separately. Then ask which triple occurs twice and which documents have more than one version.

Exercise 5.2

F02 and F03 share the same matter, document, and version labels. Why does this leave open whether their bytes are identical, and what check would settle the question?

Hint. The program reads a register of labels. It never opens the files the labels describe.

Exercise 5.3

Copy `documents.csv` and change F05's title to `Agreement`, which is also F01's title. Predict the summary and catalogue, and explain your reasoning.

Hint. Is the title part of any grouping key?

Exercise 5.4

Copy `documents.csv` and change F07's version to `final`. Predict the summary, M02's catalogue entry, and the contents of `review.csv`.

Hint. A version must be a whole number written in digits. A row that fails a check is left out of every count; D04 still has another file.

Exercise 5.5

Copy `documents.csv` and change F08's document ID to `D01`, so that M03 also has a document labelled D01. Predict the summary. What answer would a program give if it grouped by document ID alone, and how does this edit test the grouping rule?

Hint. The document key is a pair, and M03/D01 is not M01/D01. Without the matter, F08's `D01`, version 1 collides with F01's.

Chapter 6

Check a Document Register

Exercise 6.1

The F05 record on line 6 has the correct values. Why must it still be held along with the other occurrence of F05?

Hint. Which row is the real F05? Ask what would decide that if only one were held, and compare line 9 with the clean register.

Exercise 6.2

A source-reference field is nonblank. What, exactly, has the checker established? What further check would establish that the named file exists, and what would that check need?

Hint. A populated pointer is not the same as a successfully opened target.

Exercise 6.3

Copy `documents.csv`, change the file ID on the last line back to `F08`, and leave everything else alone. Predict which records will be accepted and which held.

Hint. Look at the `issues` cell of each F05 row. Was the duplicate the only problem on either of them?

Exercise 6.4

Copy `documents.csv` and change F03's type to `recorded as executed`. Predict `summary.json` and `consistency.csv`. Explain why the accepted and held counts stay the same.

Hint. F02 and F03 form one group. The consistency check runs over every row, held or not, and reports without holding.

Exercise 6.5

Copy `documents.csv` and remove F06's type. Predict its entry in `review.csv` and the summary. What does the held count tell you about records with several issues?

Hint. One record can carry several reasons. Missing cells are checked before the version.

Chapter 7

Prepare a Limited Management Extract

Exercise 7.1

Why does the extract have eight rows, and why do three of them have a count of zero? Name those three rows.

Hint. Multiply the represented weeks by the declared statuses. Then look for the combinations that no appointment fills.

Exercise 7.2

Explain why A05 counts as completed here but is absent from Chapter 4's documented-completion numerator.

Hint. Compare a question about a status label with a question about a dated event.

Exercise 7.3

You cannot find the canary on the displayed page. What two further checks should you make before accepting the extract, and what could each of them catch?

Hint. Consider what a browser does not display, what the canary does not cover, and what a count can lose.

Exercise 7.4

Copy `appointments.csv` and change A08's scheduled date to `2026-09-14`. Predict `output_rows`, `total_count`, and the entries for the week beginning September 7.

Hint. September 14, 2026, is a Monday. Every represented week gets four rows.

Exercise 7.5

Copy `appointments.csv`, change A07's status to `rescheduled`, and run the program. What happens? Explain the wording of the error message, and say why the program should stop rather than add an "other" row.

Hint. An error message is also an output. Ask where it might be pasted.

Chapter 8

Match Requests and Responses

Exercise 8.1

Which requests have a selected response under the fourteen-day rule? For each response linked to R01, explain why it is selected or left unselected.

Hint. Match both key fields first, then test elapsed days, then compare dates. Day 14 is inside the window.

Exercise 8.2

Predict the `matched` count and each request's selected response with `--window 21`, then run the program. Why does this answer a different question rather than correct the fourteen-day result?

Hint. Only one baseline candidate lies between day 15 and day 21.

Exercise 8.3

S06 has R04's matter and request ID, yet R04 says `no` for `any_response`. Explain why, and say why R04 is marked `needs_review` rather than `window_open`.

Hint. Read S06's `elapsed_days` and `valid` columns. Then count the days from R04's request to the reporting date.

Exercise 8.4

Predict the explanations for S02 and S03, R02's status, and the `matched` count with `--as-of 2026-09-10`, then run the program.

Hint. A response dated after the reporting date is kept but not yet observed. Count the days from R02's request to September 10.

Exercise 8.5

Design a test that catches matching by request ID alone even when the headline count is right. Copy `responses.csv` and change S05's date to `2026-09-03`. Predict the reference results for R03 and S05. Which response would a matcher using only request ID select for R03?

Hint. With the new date, S05 would be day 1 for R03, earlier than S04, if it were allowed to link.

Chapter 9

Define the Matters in a Report

Exercise 9.1

Classify all eight candidates on September 15, naming the matters in each group. Explain every exclusion and every decision to keep a matter for review.

Hint. Compare each opening and closure date with September 15. A date that is not a date, or a closure before an opening, cannot be compared at all.

Exercise 9.2

How many observations support the mean delay of 6 days? Calculate the mean and median if M03's blank delay were treated as zero. Why would those figures misstate the record?

Hint. Three matters are included, but how many delays were recorded? Recompute with the values 4, 0, and 8.

Exercise 9.3

Copy `population.csv` and add a ninth record: `M09,2026-09-12,,,POP09`. Predict the summary. Now add the complete record `M09,C09,Synthetic matter` to a copy of `matters.csv` and predict again. Does adding an included matter with no delay necessarily lower the mean?

Hint. The rule checks the matter register before it looks at the dates. A blank delay contributes no value to the mean.

Exercise 9.4

Run with `--as-of 2026-09-05`, the day M02 closes. Before running, predict the included matters, mean, and median. Which matter changes classification if you use `--as-of 2026-09-04` instead?

Hint. A matter closed on the reporting date is not active at the end of that day. Check also which matters had opened by September 5.

Exercise 9.5

Copy `population.csv` and change M06's `response_days` from `8` to `8.5`. Predict its `observation` value and list every summary line that changes. Why does the program leave the value unrounded?

Hint. The rule accepts only a whole number of days. An entry that is neither blank nor a whole number is invalid, not missing.

Chapter 10

Report Monthly Turnaround

Exercise 10.1

January and February have the same mean. Why do their medians differ? Name the pair that causes the difference.

Hint. Sort each month's three values. Compare the middle value with the sum divided by three.

Exercise 10.2

How might a report covering only answered requests mislead someone who wants to understand all requests, including those still outstanding?

Hint. Ask which requests never entered `pairs.csv`, and which requests had the least time to be answered before the cutoff.

Exercise 10.3

Copy `pairs.csv` and change only PAIR06's `elapsed_days`, from `20` to `2`. Predict its classification and reason, February's row, and the total observation count. Why should the program check the dates instead of trusting the supplied delay?

Hint. The dates still say February 3 to February 23. Which of the two sources does the rule check against the other?

Exercise 10.4

Run with `--as-of 2026-02-22`. First predict which pairs become `after_as_of`, what February's row shows, and what appears for March through December. What does February's result tell you about comparing a recent month with older months?

Hint. A pair is observable only if both its dates are on or before the cutoff. Look at PAIR06's response date.

Exercise 10.5

Copy `pairs.csv` and add a second pair for REQ02:

```
PAIR17,M01,REQ02,2026-01-02,2026-01-12,10,PAIR-SOURCE-17
```

Predict PAIR02's and PAIR17's classifications, January's row, and the observation count. What would a test miss if it compared only monthly means?

Hint. Two pairs for one matter and request are both held. Compute the mean of the January values that remain.

Chapter 11

Monitor Weekly Requests

Exercise 11.1

Calculate both the average weekly rate and the pooled rate for W02 through W10. Use the results to show that W01 alone causes the difference between 55% and 51.1%.

Hint. Without W01, what rate does every remaining final week have?

Exercise 11.2

What should the headline show if every final week has zero eligible requests? Test your answer in a copy of `weeks.csv`: set both `eligible` and `responded` to 0 for W01 through W10.

Hint. Look at the summed final denominator. Provisional weeks do not enter it.

Exercise 11.3

Copy `weeks.csv` and set W01's `eligible` and `responded` to 0. Predict its weekly rate, both final totals, and the pooled rate. How can a week with no rate still contribute to a pooled result that is defined?

Hint. Pooling adds counts. Adding zero to both sums is allowed; dividing zero by zero is not.

Exercise 11.4

Copy `weeks.csv` and change W12's `responded` from 2 to 12. Predict the summary. W12 is provisional and contributes nothing to the headline; why does this error suppress the headline anyway?

Hint. Twelve responded requests out of ten eligible is impossible. Reread the last item of the rule.

Exercise 11.5

Create a two-week `weeks.csv` whose average weekly rate differs from its pooled rate by at least 20 percentage points. Give the file and the expected printed summary, and identify the summary line your test checks.

Hint. Make one week very small with a high rate and the other large with a low rate. Both weeks must begin on a Monday and be final.

Chapter 12

Build a Production Index

Exercise 12.1

Change only D07's page count to 2. Predict `supplied_page_count_total`, `claimed_numbers`, and the accepted count. Does the opening "totals reconcile" check pass? Is the production more complete?

Hint. Only one of the four discrepancies is removed.

Exercise 12.2

Why should the program hold both D05 and D06 for review, rather than only the one that appears second?

Hint. Which range is wrong?

Exercise 12.3

If D01 is held, do the stated rules also hold its attachment D02? Would a stricter family rule do so? Explain the difference.

Hint. Read the family rule exactly.

Exercise 12.4

Copy the register and change D03's `bates_end` to `XYZ000007`. Predict its status and reason. What changes in the gap list?

Hint. A range with two prefixes is not a range.

Exercise 12.5

Copy `production.csv` and append this complete eleventh record:

```
D11,M01,VOL001,ABC000013,ABC000014,2,D11,letter,PRD-S11
```

Predict the gap list, the accepted count, and `claimed_numbers`.

Hint. D11 fills the hole exactly.

Chapter 13

Prepare Administrative Letters

Exercise 13.1

Account for all six requests using only `manifest.csv` and the file list in `Run_01`. What would five letter files in that folder tell you?

Hint. Every request is either generated, with a file named in the `output` column, or held, with a reason. Compare the files with that column, not with the number of rows.

Exercise 13.2

Copy `letter_template.txt` and append `Please bring any documents already requested.` Predict the summary counts and name the letters that change. Does an approval tied to the fingerprint beginning `453b974a` cover this template?

Hint. The fingerprint identifies the exact bytes of the template. Ask whether the recorded approval identifies the text now proposed for use.

Exercise 13.3

Copy `letters.csv`. Add a seventh data record by copying L01, then changing its identifier to `../L07` and recipient to `Example Recipient 7`. Predict the manifest entry. Will the program write any file outside its output folder?

Hint. Which characters may a letter identifier contain? A value in the register should never decide where a file is written.

Exercise 13.4

In a copy of `letters.csv`, set L06's time zone to `EST`. Predict the result. Why should the program keep this value rather than translate it to `America/Toronto`?

Hint. Is Toronto on standard time on September 25? Compare 14:00 EST with 14:00 in Toronto on that date.

Exercise 13.5

Copy `letters.csv` and add a seventh data record by copying L06. Keep its identifier L06 and change only its recipient to `Example Recipient 7` and its matter to M01. What happens in a simple generator that uses identifiers as filenames? What does the supplied program do?

Hint. Two letters would be written under the same file name. Which one survives? Then reread the rule about repeated identifiers.

Chapter 14

Track Professionally Supplied Dates

Exercise 14.1

Name the six baseline classifications, and for each entry that needs review, give the issue that caused it.

Hint. Apply the completeness checks to every entry before you look at any calendar position. Read D04's date against the number of days in September, and read D05's `verifier` cell.

Exercise 14.2

Before running `--as-of 2026-09-25`, predict every classification and every summary count that will change; then run it and compare.

Hint. The entries do not change; only the date they are compared with does. Check first that the verification dates, September 20, are still no later than the new reporting date.

Exercise 14.3

Copy `dates.csv` and mark D01 `superseded`. Add D07 for M01 and TASK01, with supplied date 2026-09-28, source DATE-SOURCE-07, verifier Reviewer Example, verification date 2026-09-22, owner A, completed `no`, status `current`, and `supersedes` D01. Predict the register. Then, in a fresh copy, add the same D07 but leave D01 unchanged. Why do the results differ?

Hint. Count the `current` entries for M01 and TASK01 in each version of the file. Then check what the supersession rule requires of the entry that D07 names.

Exercise 14.4

Copy `dates.csv`, set D05's `completed` value to `yes`, and predict its classification. In a fresh copy, set D06's value to `Yes`, with a capital letter, and predict its classification and the summary counts.

Hint. `recorded_complete` is reached only by an entry that has passed every check. The completion flag must be exactly one of two values.

Exercise 14.5

Write two or three checks that use named entries. Each must pass for the reference and fail for the date-first answer in *A Plausible Wrong Answer*.

Hint. Look for the three entries on which the two answers disagree, and for a count that a skipped row would change.

Chapter 15

Search Names and Parties

Exercise 15.1

Use `--exact-only` with the default query. Which candidates appear, and which columns show that they are separate entities?

Hint. Does the default query need the one-edit rule at all? Compare the surviving rows column by column, and ask which columns a name alone could never show.

Exercise 15.2

Try `A. Morgan` and `A Morgan` without the period, each with and without `--exact-only`, predicting each candidate's match type before you run. Why is N02 absent from every result?

Hint. Normalization keeps punctuation. Count the characters that separate `a morgan` from `a. morgan`. Then read N02's `aliases` cell.

Exercise 15.3

Search for `Morgan`. Why is the result empty? Describe it accurately in one sentence, without treating it as a clearance.

Hint. The query is compared with each whole name, not searched for inside it. Your sentence should name the query, the rule, and the register.

Exercise 15.4

Search for `MAPLE  SERVICES  LTD`, in capitals and with two spaces between words, and then for `Maple Services`. Predict the candidates, match types, and matched text for both queries.

Hint. Apply the normalization to the first query. For the second, compare it with N03's primary name and with its alias separately.

Exercise 15.5

Copy the chapter folder and add N05: Jordan Example, prospective contact, matter M05, source ENTITY05, with no alias. Run the default search again. Which values in `summary.json` and `candidates.csv` change, and why does the changed value matter when you examine a search saved last month?

Hint. N05 does not match Alex Morgan, so look at the summary line that describes the register rather than the result.

Chapter 16

Organize Legal Authorities

Exercise 16.1

Account for all eight retrieved records: name the records behind the five authorities, the six authority-versions, and the one ungrouped entry.

Hint. First separate the record that cannot be grouped. Then list the distinct authority IDs among the rest, and then the distinct pairs of authority ID and version.

Exercise 16.2

Why must DEC01 and DEC02 stay separate? Name the values in `membership.csv` that a test should check, and say what summary counts merging them would produce.

Hint. Read Q03's `relationship` cell. For the counts, imagine Q03 carrying DEC01 as its authority ID.

Exercise 16.3

Q07 is grouped while its verification is pending. Copy `authorities.csv`, set its `verification` to `recorded`, and leave its reviewer empty. Predict the outputs. Why does that field value alone fail to establish verification?

Hint. Which columns does the grouping rule require? The register's verification columns are carried through, not checked.

Exercise 16.4

Copy `propositions.csv`, change P02's version from 2024 to 2026, and predict its issues. In a fresh copy, change the version to 2025 and predict again. What does the first result leave unestablished?

Hint. The worksheet checks only that the authority-version exists in the register. Which versions of LEG01 does the register contain?

Exercise 16.5

Copy `authorities.csv` and give Q08 the authority ID DEC01. Predict the four summary counts and the records in DEC01's group. Who made the decision to treat Q08 as a copy of DEC01?

Hint. Q08's version is 2026. Does DEC01 already have that version?

Chapter 17

Compare Document Versions

Exercise 17.1

For P01–P04, list each changed word or number and give each paragraph's status.

Hint. Read the two files line by line, matching identifiers. In P03, remember that the comma separates the digits into two units.

Exercise 17.2

Explain why "no change in the extracted body text" says less than "no change in the complete source document".

Hint. List what a real agreement file can contain besides the body paragraphs, and ask which of those an extraction step might leave out.

Exercise 17.3

Copy the chapter folder and move P04's line to the top of `revised.txt`, keeping all text unchanged. Predict every paragraph's status and `position_changed` value, then the summary counts.

Hint. Status depends on text matched by identifier; position is the line number. Moving one line changes the line numbers of the lines it passes.

Exercise 17.4

Copy the chapter folder and, in P04 of `revised.txt`, put two spaces between *keep* and *a*. Predict the summary for an exact comparison, then for a run with `--whitespace`. In the second run, what revised text does `comparison.csv` show for P04? Why can the report still mark the whitespace difference although the status is `unchanged`?

Hint. In the exact comparison every character counts, including a space. The normalized comparison changes how texts are compared, not what is printed.

Exercise 17.5

Copy the chapter folder and add a fifth line to `revised.txt`: P05, a tab, then "The parties may sign separate copies." Predict P05's row in `comparison.csv` and the summary counts. Which paragraphs now contribute to the `changed` count?

Hint. P05 has no line in the original. Check how the rule names such a paragraph, and what position number the table gives a file in which the paragraph is absent. Make sure your editor saves a real tab, not spaces.

Chapter 18

Check Time Entries and Totals

Exercise 18.1

Calculate the baseline total and all three matter subtotals by hand. List the entries in each subtotal.

Hint. Use only the four calculable entries, then group them by the `matter_id` column.

Exercise 18.2

Copy the input file and add a seventh data record identical to T02. Predict the calculable total, the held count, and M01's subtotal. Explain why simply removing the second T02 would be the wrong repair.

Hint. A repeated identifier holds *every* occurrence. Which calculable amount leaves the total?

Exercise 18.3

Copy the input file and enter `0` for T04's minutes. Predict the summary, and explain why the original blank cannot be treated as that zero.

Hint. Which count can change when the total cannot? Decide that on paper before you run anything; the blank and the zero fail different checks.

Exercise 18.4

Copy the input file and change T05's currency to `USD`. Predict its ledger line, the total, and `matter_totals.csv`. Explain what you can and cannot conclude when M03 is absent from that file.

Hint. Look at the `currency` rule, and remember that `matter_totals.csv` sums only calculable amounts.

Exercise 18.5

Design a two-row input that gives three different totals under these methods: the reference rule; half-even rounding on each line; and rounding only after adding the unrounded amounts. Give all three totals.

Hint. Use the half-cent line of the chapter's experiment twice. Work out each convention separately.

Chapter 19

Review Files Recorded as Closed

Exercise 19.1

Account for all five register entries by category. For each entry, name the step that determines its category.

Hint. Apply the chapter's three classification steps in order; stop at the first one that decides.

Exercise 19.2

In a copy, give F01 a September 24 review date; it is complete and has no hold. Predict its category in a September 24 report, then test it.

Hint. Read step 3 again: “on or before”.

Exercise 19.3

Copy the input and remove F03’s review date. Predict its category, its `issues`, and the summary. Explain which information must remain visible and why.

Hint. Which step comes first, the hold check or the completeness check?

Exercise 19.4

Predict every category for reporting date 2026-07-31. Then run the supplied program on the unchanged files with `--as-of 2026-07-31`. Explain F03’s result.

Hint. On July 31, when had these files been closed according to the register?

Exercise 19.5

Copy the input and change F03’s hold flag from `yes` to `Yes`. Predict its output line and the summary, and explain why the program takes this cautious approach. What must someone reading the summary notice?

Hint. Is `Yes` exactly `yes`? Then look at which step applies, and at the `hold` count.

Chapter 20

Test the Applications

Exercise 20.1

The test `test_24_large_selected_identities` requires four selected events for M0001. Describe a wrong selection that a count-only check would still pass. What assertion would reject it?

Hint. How many events does each large matter have, and in what order are they numbered?

Exercise 20.2

The test `test_07_bad_request_holds_dependent_response` removes E01's source reference from the clean event file. Give the accepted and held counts, and the reason for holding each held record. Why would checks of each record's own fields miss one of the problems?

Hint. Run `matter_check.py` on a copy of the clean file with E01's source cell emptied, and read `review.csv`.

Exercise 20.3

What does the endpoint mutation establish, and what remains unchecked? Consider the matcher's other inclusive comparison, the one that decides whether a request's window is still open. You can answer from the written rule and saved failure log; rerunning the mutation is optional.

Hint. Read the saved failure log first. Ask what the failing tests have in common, and which inclusive comparison none of them exercises.

Exercise 20.4

Work on a copy of the release folder, either locally or through an assistant that can edit and execute uploaded programs. In `programs/legal_apps.py`, change the time-entry calculation from `rounding=ROUND_HALF_UP` to `rounding=ROUND_HALF_EVEN`, and add `ROUND_HALF_EVEN` to the `from decimal import` line. Predict which tests fail and which baseline test still passes. What `rounding` label does the faulty program print?

Hint. Which tests involve an amount exactly halfway between two cents? Is the `rounding` label computed or written?

Exercise 20.5

Run this locally from the release folder, or ask an assistant with the complete release to run it there:

```
python3 -m unittest discover -p "test_*.py"
```

Do not enter `cd programs` or add `-s programs`. What does the command print, and why does "no failures" alone fail to establish that the suite passed?

Hint. Discovery searches from the current folder. Read the line that says how many tests ran.

Chapter 21

Package a Reproducible Demonstration

Exercise 21.1

Group the demonstration archive's twelve members by role. Which member is absent from the manifest, and why?

Hint. Count by folder: `programs`, `data`, the top level, and what the maker adds.

Exercise 21.2

Extract the archive into a new empty folder, delete `programs/matter_tools.py` there, and run the tests. What happens? Why might tests run in the release folder miss a package that omitted this file?

Hint. Where would Python find `matter_tools.py` if you ran the same command in the release folder?

Exercise 21.3

Three manifest entries have the same fingerprint. Is that a problem? What does a matching fingerprint tell you about a received file, and what does it leave unanswered?

Hint. Open the three files. Then separate what a fingerprint says about bytes from what it says about people.

Exercise 21.4

Run the package maker locally, or ask an assistant that can execute it to make two archives. Build the demonstration twice under different names, at least three seconds apart. Compare the SHA-256 fingerprints of the two ZIP files, then of their `MANIFEST.json` members. Explain what you find. In what sense is this package reproducible?

Hint. A ZIP archive records more than the members' contents.

Exercise 21.5

Run the maker with `--root Chapters/chapter01` and a new output name. Predict its message and whether it creates an archive. Why is this better than including whatever matching files it can find?

Hint. Look at what `Chapters/chapter01` contains, and when the maker checks its list.

Chapter 22

Check Application-Review Records

Exercise 22.1

Which three reviews are complete on September 24? Give the issue code for each of the other two and say what action it calls for.

Hint. Check the artifact-version column against the declared version 2, then look for an empty field. The two problems need different people to act.

Exercise 22.2

Leave the register unchanged and run with `--as-of 2026-09-25`. Which record changes? Give its issue code and name the records that remain complete.

Hint. Only one expiry date lies between September 24 and September 25. The two existing attention items do not depend on the date.

Exercise 22.3

Leave the register unchanged and run with `--rule 2`, as though the rules had changed. Predict the summary and each row's issues. Which record should a reviewer examine first, and why?

Hint. Every entry records the rule version it reviewed.

Exercise 22.4

Copy `reviews.csv` and add a sixth row: `program,2,1,Reviewer Example, 2026-09-20,2026-12-31,REVIEW-6`. Predict both program rows' statuses and issues, and the summary counts. Why does the newer-looking entry fail to make the program category complete?

Hint. The rule requires exactly one row per category. A row can carry more than one issue.

Exercise 22.5

Run the unchanged register with `--artifact 3`, as though version 3 were being delivered. Predict the complete and needs-attention counts and the data-handling issues. Then consider `--artifact 1`: which record, if any, is complete?

Hint. Compare the declared artifact version with each row's `artifact_version` column before counting anything.

Chapter 23

Build an Integrated Matter Worklist

Exercise 23.1

Give each of the six baseline requests its status and three evidence fields. Why must Q06 retain its response rather than be labelled unanswered?

Hint. Apply the chapter's status order to each request, then look up CS06 in `candidates.csv`.

Exercise 23.2

Predict the complete worklist for `--as-of 2026-09-25` before running the supplied program. Which request changes status? Which unanswered request stays open?

Hint. Add one day to the age of each unanswered request. Which one was exactly at the fourteen-day boundary?

Exercise 23.3

Copy `responses.csv` and add `CS02,M01,Q01,2026-09-12,CR02`. Give Q01's status, three evidence fields, and selected response. Give CS01's and CS02's selection values in the candidate ledger. What changes in the summary?

Hint. CS02 has the same date as CS01. Existence of an in-window response is not the same as a unique earliest one.

Exercise 23.4

Run the unchanged files with `--matter M02`. Predict the rows in `worklist.csv` and `candidates.csv`, the summary counts, and `internal_validation.json`. Which files should go to a recipient for matter M02?

Hint. Look at which requests and responses name M02. The internal record describes the whole input, not the selection.

Exercise 23.5

Copy `assignments_large.csv`, remove M0002's record, and run the conversion. What does it print? Which files does it create? Why is this better than giving M0002's request a blank owner?

Hint. The assignment table is checked against the matter register before anything is written. Think about what an owner filter would show for a blank owner.

Part II

Solutions

Chapter 1

Build a Matter Timeline

Exercise 1.1

List the event identifiers for M02. Why must they stay out of the display when you select M01?

Solution. M02's events are E04, E05, and E06. They share client C01 with M01's events, but the timeline is organized by matter. Selecting M01 must show exactly E01, E02, and E03; showing six events would mean the page filtered by client.

Exercise 1.2

Copy the file and replace E03's summary with `A & B <b>review</b>`. What exactly should the page display? Explain why the angle brackets must remain visible and the word `review` must not become bold.

Solution. The page must display the characters `A & B <b>review</b>` literally, in the event details and in the table. The angle brackets and both tag-shaped strings must be visible; the word *review* must not become bold because of the supplied tags. In the HTML source, the ampersand is escaped as `&` and the angle brackets as `<` and `>`. If the tags were inserted unescaped, the browser would interpret them as markup. Source text must never become part of the page's structure. The record counts do not change.

Exercise 1.3

Design a test to catch a timeline that selects by client instead of matter. State the matter to select, the identifiers you expect, and one identifier that must not appear.

Solution. Select M01. The expected identities are exactly E01, E02, and E03. E04 (and E05, E06) must be absent, because they belong to M02, the other matter of the same client C01. A client filter would return six events and fail the test; a count of three alone is not enough, so the test compares identities.

Exercise 1.4

Open your run's `accepted.csv`. Which column points back to the input record? Find its value for E05 and explain why it is not 5.

Solution. The `source_row` column. E05 has source row 6: the header line is row 1, and E05 is the fifth record, so it is on row 6. The value is a logical record position, not the event number; it remains correct even when identifiers are not numbered in file order.

Exercise 1.5

Change one letter in E03's summary in a copy of the input, then run again. Which lines in the run record change? Which stay the same? What does this show about fingerprints?

Solution. Keep the input's filename unchanged. Only `input_sha256` changes in the run record. The counts (18 input, 18 accepted, 0 held), the register fingerprint, the program fingerprint, and the version stay the same. A one-letter change in one file changes that file's fingerprint completely and leaves the others untouched, so the run record identifies exactly which input differed between two runs. It does not say which letter changed or whether the change matters; for that you compare the files themselves.

Chapter 2

Check the Matter Records

Exercise 2.1

Copy the problem file. Restore E14's matter to M05 and E17's client to C03. Change nothing else. Predict the accepted count, held count, held identifiers, and number of reasons.

Solution. Thirteen accepted and five held: E02 (`response_before_request`), E05 and E08 (`invalid_or_missing_date`), and both E10 rows. The second E10 still has two reasons, so there are six reasons on five rows. E14 and E17 are accepted again, and E14's second reason disappears with its first, because its request M05/R05 now exists.

Exercise 2.2

Design a test to distinguish matching by client from matching by matter and request. State the input edit and the record that should be held.

Solution. In a copy of the clean file, change E02's matter from M01 to M02, keeping client C01 and request R01. A matcher keyed on client and request finds R01 for C01 and accepts E02. The reference looks for request R01 in matter M02, finds none, and holds E02 with `missing_or_ambiguous_request`: 17 accepted, 1 held.

Exercise 2.3

These rules hold an otherwise valid event whose summary is blank; another office might accept it. Does that make this checker wrong? What would need to change?

Solution. The checker is not wrong: it implements the stated rule, which requires a nonblank summary. An office that accepts blank summaries has a different specification. Changing the behaviour is a rule change, so it needs a new written rule, new expected results, and updated tests—not a quiet repair of the program.

Exercise 2.4

Copy the problem file and leave its existing problems in place. Remove SRC01 from E01. How many records should be held? What reasons should E01 and E02 have?

Solution. Eight records are held and ten accepted. E01 is held with `missing_source_id`. E02 is held with `request_needs_review`: its request is now itself held, so the response cannot be placed on it. (E02 was already held for its early date; once its request is invalid, the reference reports the dependency instead, because the date comparison needs a valid request.) The other six held records are unchanged.

Exercise 2.5

An assistant reports "12 accepted, 6 held" on the unchanged problem file, and you cannot see its program. Which review row would you inspect first, and what would you expect to find?

Solution. Inspect the E10 rows (source rows 11 and 12). The likely explanation is that the assistant kept one E10—usually the first—and held only the other, letting file order decide an identity dispute. Its review file would then contain one E10 instead of two. Ask for the rule it used for duplicates and for the identities of all held rows.

Chapter 3

From Eighteen Records to 12,000 Events

Exercise 3.1

Count the events for client C0001 and for matter M0001. List the identifiers in each group.

Solution. C0001 has twelve events, E00001–E00012, across M0001, M0002, and M0003. M0001 has four, E00001–E00004.

Exercise 3.2

Why does a total of 12,000 events fail to show that every event belongs to the right matter?

Solution. The total counts rows. A program that assigned events to the wrong matters, or a file in which two matters' events were swapped, could still total 12,000. Correct assignment needs identity checks: named matters with their expected events, the client–matter relationship against the register, and request–response links.

Exercise 3.3

Run the irregular file. Identify the three held records and the reason for each.

Solution. Eight are accepted and three held: I008 with `response_before_request`, I009 with `invalid_or_missing_date`, and I010 with `missing_or_ambiguous_request` (its request identifier UNKNOWN matches no request).

Exercise 3.4

Run the irregular file with `--matter I04`, then with `--matter I05`. Describe both results. Why must they differ?

Solution. With I04 the run succeeds: whole-file counts 8 accepted and 3 held, selected counts 0 and 0, and an empty report. I04 is in the register, so "no events" is a true statement about a known matter. With I05 the program stops with `Unknown selected matter: I05` and writes no report: an empty page would falsely suggest that I05 is a matter with no events.

Exercise 3.5

Copy the large file and change E00010's identifier to E00002. Predict the accepted and held counts for the whole file and the selected counts for M0001. Which records should be held? Why should the selected report change?

Solution. Four records are held and 11,996 accepted. Both E00002 rows are held as duplicates: the original request R0001 in M0001 and the former E00010, request R0003 in M0003. Their responses E00003 (M0001) and E00011 (M0003) are held with `request_needs_review`. The selected M0001 report shows 2 accepted (E00001, E00004) and 2 held (E00002, E00003). A change made in another matter altered M0001's report because identifiers are unique across the whole file, which is exactly why validation precedes selection.

Chapter 4

Check Consultation Completion

Exercise 4.1

Consider cancelling one appointment at a time. Which cancellations would raise the percentage, and to what? Which would lower it? Explain your predictions, then run one case to check.

Solution. Cancelling A04, A05, or A07 removes a denominator-only appointment: 3 of 5, 60%. Cancelling A01, A02, or A06 removes a documented completion from both counts: 2 of 5, 40%. A03 and A08 are already cancelled, so nothing changes (50%). The exercise shows how much a single status label can move a small percentage, and why the label's source matters.

Exercise 4.2

What percentage should appear if every appointment is cancelled? Explain your answer.

Solution. "Not available." Every appointment is excluded, so the denominator is zero and no proportion was observed. The reference reports numerator 0, denominator 0, percentage unavailable, and `headline_available: false`. Printing 0% would claim an observed rate of zero.

Exercise 4.3

Set A05's completion date to 2026-09-04, one day before its scheduled date. Predict its ledger entry and the resulting percentage.

Solution. A05 stays in the denominator (eligible 1) but not in the numerator (numerator 0), with explanation `completion_date_needs_review`. The result is unchanged: 3 of 6, 50%. The date is present but impossible under the rule, so it does not document a completion.

Exercise 4.4

Copy A08's complete row and append the copy as a ninth appointment. In the copy, change only its status to `completed` and its completion date to `2026-09-08`; keep the appointment ID A08 and all other values. What should happen to the headline percentage? Why is withholding it better than calculating it?

Solution. Both A08 rows are flagged `duplicate_appointment_id`, and the headline is suppressed: `percentage` is unavailable and `headline_available` is false, although the ledger still lists all nine rows. Which A08 is real decides whether the appointment was cancelled or completed, so any percentage would depend on an unresolved identity. A precise-looking rate would hide that.

Exercise 4.5

Before running with `--as-of 2026-09-05`, predict the numerator and denominator. Which appointments leave the denominator, and why? Run the program to check.

Solution. Numerator 2 (A01, A02) and denominator 4 (A01, A02, A04, A05): 50%. A03 is cancelled and excluded. A06, A07, and A08 are scheduled after September 5, so the report does not cover them: all three are labelled `not_yet_scheduled` with `eligible` 0, whatever their other cells say, and A06's September 6 completion date is not examined, because a report as of September 5 has no business judging it. The summary therefore shows `cancelled` 1 (A03 only; A08's cancellation lies outside the report) and `unresolved` 1 (A05, still without a completion date). A05 is scheduled on the reporting date itself and stays in the denominator. The same records give a different, equally correct answer to a different historical question, and the counts describe only the appointments the question covers.

Chapter 5

Catalogue Matter Documents

Exercise 5.1

The same input gives totals of eight, five, and seven. Explain each total, naming the records that account for the differences.

Solution. Eight file records: F01 to F08, one per row. Five documents (matter and document ID): M01/D01, M01/D02, M02/D03, M02/D04, M03/D05. Seven labelled versions (matter, document ID, and version): M01/D01/1, M01/D01/2, M01/D02/1, M02/D03/1, M02/D04/1, M02/D04/2, M03/D05/1. Files exceed versions by one because F02 and F03 are two recorded copies of M01/D01/2. Versions exceed documents by two because D01 (F01; F02 and F03) and D04 (F06; F07) each have two versions. By matter: M01 has 4, 2, 3; M02 has 3, 2, 3; M03 has 1, 1, 1.

Exercise 5.2

F02 and F03 share the same matter, document, and version labels. Why does this leave open whether their bytes are identical, and what check would settle the question?

Solution. The catalogue reads only the register: labels that a person entered to describe how files are grouped. It never opens the files. Even the register hints at a difference, since F02's type is `draft` and F03's is `draft copy`, and each has its own source reference (S02, S03). Identical bytes can be established only from the files themselves: by comparing them byte by byte, or by computing a fingerprint (such as the SHA-256 values in `run.json`) of each file and comparing the two. Equal fingerprints would show identical contents; they would still not say which copy, if either, is authoritative.

Exercise 5.3

Copy `documents.csv` and change F05's title to `Agreement`, which is also F01's title. Predict the summary and catalogue, and explain your reasoning.

Solution. Nothing in the counts changes. The summary is still documents 5, file records 8, held 0, versions 7, and `catalogue.csv` is identical (M01 4, 2, 3; M02 3, 2, 3; M03 1, 1, 1). In `membership.csv` only F05's title cell differs. The title is not part of any grouping key: F05 remains M02/D03, version 1. The catalogue does not infer identity from appearance, so two unrelated files with the same title are not merged.

Exercise 5.4

Copy `documents.csv` and change F07's version to `final`. Predict the summary, M02's catalogue entry, and the contents of `review.csv`.

Solution. The summary becomes documents 5, file records 7, held 1, versions 6. M02's line becomes `M02,2,2,2`: files F05 and F06, documents D03 and D04, versions D03/1 and D04/1. M01 and M03 are unchanged. `review.csv` has one row: F07, M02, D04, version `final`, type `draft`, source S07, source row 8, issue `invalid_version`. The document D04 is still counted, through F06. The program does not translate `final` into a number or treat it as the latest version; a person must decide what the label means.

Exercise 5.5

Copy `documents.csv` and change F08's document ID to `D01`, so that M03 also has a document labelled D01. Predict the summary. What answer would a program give if it grouped by document ID alone, and how does this edit test the grouping rule?

Solution. The summary is unchanged: documents 5, file records 8, held 0, versions 7, and M03's catalogue line is still 1, 1, 1. The key M03/D01 is different from M01/D01. A program that groups by document ID alone would find only four documents (D01, D02, D03, D04) and six document–version pairs, because F08's D01 version 1 would merge with F01's. The edit is therefore a test of the key: on this input, require 5 documents and 7 versions. The original register cannot catch a missing matter in the key, because its document labels happen to be unique across matters.

Chapter 6

Check a Document Register

Exercise 6.1

The F05 record on line 6 has the correct values. Why must it still be held along with the other occurrence of F05?

Solution. The rule gives `duplicate_file_id` to every occurrence because the program cannot tell which row is entitled to the identifier. Line 6 (M02, D03, source S05) and line 9 (M03, D05, source S08) both claim F05. Accepting the first would let the order of the file decide an identity question. Here that would happen to pick the right row, since the clean register shows that line 9 was F08, but only by luck: had the rows been in the other order, the same rule would have accepted the wrong record. Holding both puts the question to a person.

Exercise 6.2

A source-reference field is nonblank. What, exactly, has the checker established? What further check would establish that the named file exists, and what would that check need?

Solution. The nonblank check asks only whether the cell contains something. `S03` passes it, and so would a mistyped reference or one naming a file that was never saved. An existence check must resolve the reference to an actual location and confirm that the target is there and can be opened. That needs a defined storage root, a rule for turning a reference into a permitted path, and access to the storage; a stronger version would also compare a recorded fingerprint, to show that it is the right file. The supplied program performs only the first check.

Exercise 6.3

Copy `documents.csv`, change the file ID on the last line back to `F08`, and leave everything else alone. Predict which records will be accepted and which held.

Solution. Accepted 5: F01 (row 2), F02 (row 3), F05 (row 6), F07 (row 8), F08 (row 9). Held 3: F03 (`missing_source_id`), F04 (`unknown_matter`), F06 (`invalid_version`). The summary shows input rows 8, accepted 5, held 3, consistency rows 0. One edit releases two rows, because the duplicate was the only issue on each F05 row.

Exercise 6.4

Copy `documents.csv` and change F03's type to `recorded as executed`. Predict `summary.json` and `consistency.csv`. Explain why the accepted and held counts stay the same.

Solution. The summary is accepted 3, consistency rows 2, held 5, input rows 8. `consistency.csv` has two lines, both `conflicting_supplied_types`: source row 3, F02, original type `draft`; and source row 4, F03, original type `recorded as executed`. F02 and F03 share M01/D01/version 2, and their labels now contradict each other. The counts do not change because a consistency issue holds nothing: F02 stays accepted, and F03 was already held for its missing source reference, with that reason unchanged. The check ran over the held F03 as well; a held record still provides evidence about its group.

Exercise 6.5

Copy `documents.csv` and remove F06's type. Predict its entry in `review.csv` and the summary. What does the held count tell you about records with several issues?

Solution. F06's line keeps its values (version `two`, type blank, source S06, source row 7) with issues `missing_type;invalid_version`. The summary is unchanged: accepted 3, held 5, consistency rows 0, input rows 8. `held` counts records, not reasons: there are now six reasons on five held rows. To count reasons, split each `issues` cell at the semicolons.

Chapter 7

Prepare a Limited Management Extract

Exercise 7.1

Why does the extract have eight rows, and why do three of them have a count of zero? Name those three rows.

Solution. The input covers two Monday weeks (beginning 2026-08-31 and 2026-09-07), and the specification requires one row for each of the four statuses in every represented week: $2 \times 4 = 8$. The zero rows are 2026-08-31 `scheduled`, 2026-09-07 `completed`, and 2026-09-07 `missed`. They are printed so that a reader can tell "none in the file" from "not reported". The counts sum to 8, one for each appointment.

Exercise 7.2

Explain why A05 counts as completed here but is absent from Chapter 4's documented-completion numerator.

Solution. The extract counts supplied status labels, and A05's label is `completed`, in the week beginning August 31. Chapter 4's numerator requires a completed status *and* a valid completion date between the scheduled date and the reporting date. A05 has no completion date, so there it stays in the denominator only, flagged `completion_date_needs_review`. The reports answer different questions, and each says which.

Exercise 7.3

You cannot find the canary on the displayed page. What two further checks should you make before accepting the extract, and what could each of them catch?

Solution. First, search the raw contents of every delivered file (`extract.csv`, the source of `report.html`, `summary.json`, `run.json`, and any log) for `PRIVATE-NOTE-X83`. This catches content that is present but not displayed, such as hidden columns or a debugging log beside the report. The reference gives zero occurrences in each output and one in `appointments.csv`. Second, check the columns: the first line of `extract.csv` must be exactly `week_beginning,status,count`, and the report must show no other fields. This catches names, IDs, and dates, which do not contain the canary. A third check reconciles the counts (eight rows, total 8, equal to the input rows), catching an appointment dropped or counted twice.

Exercise 7.4

Copy `appointments.csv` and change A08's scheduled date to `2026-09-14`. Predict `output_rows`, `total_count`, and the entries for the week beginning September 7.

Solution. September 14 is a Monday, so A08 starts a third week. `input_rows` is 8, `output_rows` 12, and `total_count` 8. Week of September 7: completed 0, cancelled 0, missed 0, scheduled 1 (A07). Week of September 14: completed 0, cancelled 1 (A08), missed 0, scheduled 0. The week of August 31 is unchanged. Seven of the twelve rows are now zeros.

Exercise 7.5

Copy `appointments.csv`, change A07's status to `rescheduled`, and run the program. What happens? Explain the wording of the error message, and say why the program should stop rather than add an "other" row.

Solution. The program stops before writing anything. It prints `ValueError: Invalid reporting date or status; inspect the source locally`, exits with status 2, and creates no `Run_01` folder. The message deliberately quotes neither the row nor the value: error text is often pasted into a chat or saved in a log, and it should not carry source content any more than the extract does. The person with access to the source finds the row. An automatic "other" row would give an undefined label a place in the counts, so the extract would quietly mean something new. Adding a category is a change to the specification and should be decided, not defaulted.

Chapter 8

Match Requests and Responses

Exercise 8.1

Which requests have a selected response under the fourteen-day rule? For each response linked to R01, explain why it is selected or left unselected.

Solution. R01 selects S01 and R03 selects S04 (day 6); the summary shows matched 2, ambiguous 0. R02 has no selected response: its only candidate, S03, is day 15, outside the window. R04 has none: its only linked candidate, S06, precedes it. Two responses are linked to R01. S01 is day 0, eligible, and earliest, so it is `selected`. S02 is day 14, eligible because the window includes day 14, but later than S01, so it is `not_selected` with explanation `in_window`. S05 is not linked to R01 or to any request.

Exercise 8.2

Predict the `matched` count and each request's selected response with `--window 21`, then run the program. Why does this answer a different question rather than correct the fourteen-day result?

Solution. `matched` becomes 3, `ambiguous` stays 0, and `window_days` is printed as 21. R01 still selects S01 (S02 is still later). R02 now selects S03, day 15. R03 still selects S04. R04 still has none and remains `needs_review`, because S06 precedes the request whatever the window. The fourteen-day answer was correct for its rule; the new run answers a different question. Keep both outputs and record the window with each.

Exercise 8.3

S06 has R04's matter and request ID, yet R04 says `no` for `any_response`. Explain why, and say why R04 is marked `needs_review` rather than `window_open`.

Solution. S06 links to R04 by key, but it is dated September 9, one day before the request (`elapsed_days` -1). It is therefore `response_before_request` with `valid` 0, and `any_response` counts only valid linked responses. Without S06, R04 would be `window_open`: it was requested on September 10, and the reporting date is 14 days later, still inside the window. But a linked response that cannot be right as dated takes precedence: the status is `needs_review`, with issue `linked_response_needs_review`, because a person must find out whether the response date, the request date, or the link is wrong.

Exercise 8.4

Predict the explanations for S02 and S03, R02's status, and the `matched` count with `--as-of 2026-09-10`, then run the program.

Solution. S02 (September 15) and S03 (September 18) are both `after_as_of`: `valid` 1, `observed` 0, `eligible` 0, with elapsed days still shown (14 and 15). R02, requested 7 days before the reporting date and with no observed response, is `window_open`, with `any_response` no. `matched` is 2: R01 still selects S01 and R03 still selects S04 (September 8). R04 is still `needs_review`. The ledger still lists all seven responses; the later ones are kept, not deleted.

Exercise 8.5

Design a test that catches matching by request ID alone even when the headline count is right. Copy `responses.csv` and change S05's date to `2026-09-03`. Predict the reference results for R03 and S05. Which response would a matcher using only request ID select for R03?

Solution. The reference result is unchanged: matched 2, R03 selects S04, and S05 (now dated 2026-09-03) is in `candidates.csv` and `unmatched.csv` with `unknown_request_key` and blank elapsed days. A request-ID-only matcher links S05 to R03 at day 1 and selects it, because it is earlier than S04. Its headline is also two matched requests. The test must therefore assert identities: R03's `selected_response` is S04, and S05 appears in the unmatched list. A test that checked only `matched` $= 2$ would pass both programs.

Chapter 9

Define the Matters in a Report

Exercise 9.1

Classify all eight candidates on September 15, naming the matters in each group. Explain every exclusion and every decision to keep a matter for review.

Solution. Included (3): M01 (opened September 1, no closure), M03 (opened September 3, no closure), M06 (opened September 10, no closure). Excluded (3): M02 closed on September 5 and M05 on August 31, both before the reporting date; M04 opens on October 1 and had not yet opened. Review (2): M07 has the opening date `not-a-date` (`invalid_opening`); M08 closes on September 1, before its September 2 opening (`invalid_closure`). Thus $8 = 3 + 3 + 2$, which matches `included`, `excluded`, and `review` in the summary.

Exercise 9.2

How many observations support the mean delay of 6 days? Calculate the mean and median if M03's blank delay were treated as zero. Why would those figures misstate the record?

Solution. Two observations: M01 (4 days) and M06 (8 days), with one included matter, M03, missing. The report should read "mean 6 days, median 6, from 2 of 3 included matters". Read as zero, M03 would give the values 4, 0, and 8: mean 4 and median 4. That would assert a same-day response that nobody recorded, and it would make the office look two days faster than its records show.

Exercise 9.3

Copy `population.csv` and add a ninth record: `M09,2026-09-12,,,POP09`. Predict the summary. Now add the complete record `M09,C09,Synthetic matter` to a copy of `matters.csv` and predict again. Does adding an included matter with no delay necessarily lower the mean?

Solution. With the first change only, M09 is not in the matter register, so it goes to review with `unknown_matter`. The summary is included 3, excluded 3, review 3, observations 2, missing or invalid 1, mean 6, median 6.0. Once M09 is also in `matters.csv`, it opens on September 12 with no closure and is included with observation `missing`: included 4, excluded 3, review 2, observations 2, missing or invalid 2, mean 6, median 6.0. The mean need not fall. It is computed from the

same two recorded values; only the missing count rises. It would fall only if the blank were wrongly treated as zero.

Exercise 9.4

Run with `--as-of 2026-09-05`, the day M02 closes. Before running, predict the included matters, mean, and median. Which matter changes classification if you use `--as-of 2026-09-04` instead?

Solution. On September 5, the included matters are M01 and M03. M02 closes that day and is excluded; M06 opens on September 10 and M04 on October 1, so neither has opened; M05 is closed. The summary is included 2, excluded 4, review 2, observations 1 (M01), missing or invalid 1 (M03), mean 4, median 4. With `--as-of 2026-09-04`, M02's closure is later than the reporting date, so M02 becomes included, with a missing delay: included 3 (M01, M02, M03), excluded 3, review 2, observations 1, missing or invalid 2, mean 4, median 4. The one day decides M02's membership, which is why the rule must say what a closure on the reporting date means.

Exercise 9.5

Copy `population.csv` and change M06's `response_days` from `8` to `8.5`. Predict its `observation` value and list every summary line that changes. Why does the program leave the value unrounded?

Solution. M06 stays included, and its `observation` becomes `invalid_value`; its `issues` cell stays empty, because the population rule itself is satisfied. In the summary, `observations` falls from 2 to 1, `missing_or_invalid_observations` rises from 1 to 2, and the mean and median become 4 (M01 alone). The classification counts do not change. Rounding would choose between 8 and 9 on the program's authority, and a fractional entry may be a typing error or a different unit; the value goes back to its source for correction, and the summary shows that only one observation remains.

Chapter 10

Report Monthly Turnaround

Exercise 10.1

January and February have the same mean. Why do their medians differ? Name the pair that causes the difference.

Solution. January's values are 4, 8, and 12: they add up to 24, so the mean is 8, and the middle value is also 8. February's values are 2, 2, and 20: they also add up to 24, so the mean is again 8, but the middle value is 2. PAIR06, requested on February 3 and answered on February 23, is responsible. Its 20 days raise the sum, and therefore the mean, without changing the middle value. Neither summary corrects the other; together with the count they describe the month.

Exercise 10.2

How might a report covering only answered requests mislead someone who wants to understand all requests, including those still outstanding?

Solution. `pairs.csv` contains only requests that received a selected response. A request still waiting is absent, however long it has waited, so the slowest cases are the ones most likely to be missing. Requests made shortly before the cutoff have had the least time to be answered, so only their quick responses can appear. The summary therefore states that outstanding requests are "not derivable from observed-pair input". Counting them needs the request register itself, as in Chapter 8.

Exercise 10.3

Copy `pairs.csv` and change only PAIR06's `elapsed_days`, from `20` to `2`. Predict its classification and reason, February's row, and the total observation count. Why should the program check the dates instead of trusting the supplied delay?

Solution. PAIR06 is classified `review` with `elapsed_disagrees_with_dates`, because February 3 to February 23 is 20 days, not 2. February's row becomes count 2, mean 2, median 2.0 (PAIR04 and PAIR05 only), and `observations` falls to 15. No other month changes. If the supplied value were trusted, February would show a mean of 2 from three pairs, and the chart would display a fast month built on a typing error. Because the dates and the number are two independent records of the same fact, the disagreement can be detected; the pair is preserved in `pairs_checked.csv` for correction at its source.

Exercise 10.4

Run with `--as-of 2026-02-22`. First predict which pairs become `after_as_of`, what February's row shows, and what appears for March through December. What does February's result tell you about comparing a recent month with older months?

Solution. Eleven pairs are `after_as_of`: PAIR06, whose response on February 23 comes one day after the cutoff, and PAIR07 to PAIR16, whose requests come after it. `observations` is 5. January is unchanged (count 3, mean 8, median 8). February has count 2, mean 2, median 2.0 (PAIR04 and PAIR05). March to December each have count 0 and blank mean and median; the table still has twelve rows. On February 22, February looks like a fast month only because its slow request had not yet been answered, so it was not yet in the data. A recent month's figures rest on the requests that were answered quickly; compare it with older months only after stating the cutoff, or after enough time has passed.

Exercise 10.5

Copy `pairs.csv` and add a second pair for REQ02:

```
PAIR17,M01,REQ02,2026-01-02,2026-01-12,10,PAIR-SOURCE-17
```

Predict PAIR02's and PAIR17's classifications, January's row, and the observation count. What would a test miss if it compared only monthly means?

Solution. The file now claims two different selected responses for one request, and the program does not choose between them. Both PAIR02 and PAIR17 are classified `review`, each with the reason `duplicate_selected_pair`. January keeps PAIR01 (4) and PAIR03 (12): count 2, mean 8, median 8.0. `observations` falls from 16 to 15. January's mean is 8 before and after, so a test comparing only means passes. The count (3 to 2), the total observations (16 to 15), and the two `review` lines in `pairs_checked.csv` expose the change.

Chapter 11

Monitor Weekly Requests

Exercise 11.1

Calculate both the average weekly rate and the pooled rate for W02 through W10. Use the results to show that W01 alone causes the difference between 55% and 51.1%.

Solution. W02 to W10 all have a rate of 50%, so their average is 50%. Pooled, they have $8 + 8 \times 10 = 88$ eligible requests and $4 + 8 \times 5 = 44$ responses: $44/88 = 50\%$. The two methods agree. Adding W01 (2 of 2, 100%) changes the average to $(100 + 9 \times 50)/10 = 55\%$, because W01 counts as one week in ten, but changes the pooled rate only to $46/90 \approx 51.1\%$, because it adds two requests out of ninety. Weighting matters only when weeks differ both in size and in rate; W01 is the smallest week and has the highest rate.

Exercise 11.2

What should the headline show if every final week has zero eligible requests? Test your answer in a copy of `weeks.csv`: set both `eligible` and `responded` to 0 for W01 through W10.

Solution. In the summary, both final totals are 0; `pooled_percentage` is null, shown as unavailable in the report; and `headline_available` is false. The rates of W01 to W10 are blank. W11 (80.0) and W12 (20.0) still show their rates, but they are provisional and cannot supply a final headline. The correct report gives the zero counts and says that no final rate exists; printing 0% would claim that requests went unanswered when there were none.

Exercise 11.3

Copy `weeks.csv` and set W01's `eligible` and `responded` to 0. Predict its weekly rate, both final totals, and the pooled rate. How can a week with no rate still contribute to a pooled result that is defined?

Solution. W01's rate is blank and its `issues` cell is empty: a week with no eligible requests is valid, it simply has no rate. The final totals become 88 eligible and 44 responded, and the pooled rate is 50.0%, with the headline available. W01 adds zero to each sum. Its own rate would be 0/0, which is undefined, but the pooled rate divides the sums, and the sums are positive because the other final weeks have eligible requests.

Exercise 11.4

Copy `weeks.csv` and change W12's `responded` from 2 to 12. Predict the summary. W12 is provisional and contributes nothing to the headline; why does this error suppress the headline anyway?

Solution. W12 gets `invalid_counts` and a blank rate. The summary still reports `eligible_final` 90 and `responded_final` 46, but `pooled_percentage` is null and `headline_available` is false. Under the rule, any structural problem in the file suppresses the headline, whether or not the faulty line would have contributed to it. An impossible count shows that the spreadsheet was produced or edited incorrectly, and the same fault may affect lines that look plausible. A headline labelled final should not be issued from a file known to contain an error. The counts remain visible so that the fault can be traced and corrected at its source, and the program rerun.

Exercise 11.5

Create a two-week `weeks.csv` whose average weekly rate differs from its pooled rate by at least 20 percentage points. Give the file and the expected printed summary, and identify the summary line your test checks.

Solution. One suitable file:

```
week_id,week_beginning,eligible,responded,status
W01,2026-06-29,1,1,final
W02,2026-07-06,9,0,final
```

The weekly rates are 100.0 and 0.0, so their average is 50%. The supplied program prints `eligible_final` 10, `responded_final` 1, `pooled_percentage` 10.0, and `headline_available` true. The test checks that `pooled_percentage` equals 10.0 and that the two totals are 10 and 1. An implementation that averages rates prints 50 and fails by 40 points. Both dates are Mondays; a non-Monday date would make the test fail for an unrelated reason.

Chapter 12

Build a Production Index

Exercise 12.1

Change only D07's page count to 2. Predict `supplied_page_count_total`, `claimed_numbers`, and the accepted count. Does the opening "totals reconcile" check pass? Is the production more complete?

Solution. The page-count total falls to 25; `claimed_numbers` stays 24; D07 is accepted, so seven documents are accepted and three held. The totals check now *fails*, 25 against the 26-number span, although the production is in fact slightly cleaner than before: the gap and the overlap are unchanged. A totals check can pass on a bad production and fail on a better one; it does not measure what matters.

Exercise 12.2

Why should the program hold both D05 and D06 for review, rather than only the one that appears second?

Solution. The register alone cannot say which range is wrong: D05 may end too late, or D06 may start too early. Holding only the second document would let file order decide, and would present one of the two as correct without evidence. Both are held and the overlapping label is listed, so a person can check the stamped pages.

Exercise 12.3

If D01 is held, do the stated rules also hold its attachment D02? Would a stricter family rule do so? Explain the difference.

Solution. Under the stated rules, no: the family check only requires that the parent named in `family_id` exists in the register, and D01 does. (Changing D01's page count to 5, for example, holds D01 and leaves D02 accepted.) A stricter rule, "an attachment needs an accepted parent," would hold D02 as well. That would be a different specification, with its own tests. Requiring an accepted parent also requires that the parent exists, but the existence check alone does not imply acceptance.

Exercise 12.4

Copy the register and change D03's `bates_end` to `XYZ000007`. Predict its status and reason. What changes in the gap list?

Solution. D03 is held with `prefix_mismatch`. Because its range is no longer well formed, it claims no numbers, so ABC000007 becomes a new gap. The gap list now has two entries: ABC000007 (one number) and ABC000013–ABC000014 (two). Accepted falls to five, held rises to five, and `claimed_numbers` falls to 23.

Exercise 12.5

Copy `production.csv` and append this complete eleventh record:

```
D11,M01,VOL001,ABC000013,ABC000014,2,D11,letter,PRD-S11
```

Predict the gap list, the accepted count, and `claimed_numbers`.

Solution. The gap list becomes empty, `claimed_numbers` rises to 26, and accepted rises to seven (D11 is accepted). The overlap at ABC000016 and the other held documents are unchanged, so held stays four. Filling a gap with a new record is an input change that needs a source: in real work you would add D11 only if the stamped pages exist.

Chapter 13

Prepare Administrative Letters

Exercise 13.1

Account for all six requests using only `manifest.csv` and the file list in `Run_01`. What would five letter files in that folder tell you?

Solution. The manifest has six data rows and a header. Four data rows are `generated`, with outputs `letter_L01.html`, `letter_L02.html`, `letter_L04.html`, and `letter_L06.html`, and those are exactly the four letter files in the folder. Two are `held`: L03 (source row 4, `missing_recipient`) and L05 (source row 6, `invalid_meeting_date`), with empty `output` cells. So $6 = 4 + 2$, and every file corresponds to one generated line. A fifth letter file would be one that no manifest line accounts for: perhaps a letter for a held request, or a file from another run copied into the folder. It does not belong to this run's decisions until its origin is known. The `output_hashes` in `run.json` list every file the run wrote, so the stray file can be identified by comparison.

Exercise 13.2

Copy `letter_template.txt` and append `Please bring any documents already requested.` Predict the summary counts and name the letters that change. Does an approval tied to the fingerprint beginning `453b974a` cover this template?

Solution. The summary still reports 6 input rows, 4 generated, and 2 held, but `template_sha256` is a completely different value. All four drafts change, because each now ends with the new sentence; the manifest is unchanged. The earlier approval remains attached to the template whose fingerprint begins `453b974a` and to the drafts made from it. It does not cover the new template. The new text needs its own review, recorded against its own fingerprint; a system must never transfer an approval silently to different content, however small the change.

Exercise 13.3

Copy `letters.csv`. Add a seventh data record by copying L01, then changing its identifier to `../L07` and recipient to `Example Recipient 7`. Predict the manifest entry. Will the program write any file outside its output folder?

Solution. The new request is held with `unsafe_letter_id`; its manifest line is source row 8, `../L07`, M01, `held`, an empty output, and `unsafe_letter_id`. The summary reports 7 input rows, 4 generated, and 3 held. No file for L07 is written anywhere, inside or outside the output folder, because an identifier may contain only letters, digits, hyphens, and underscores, and the characters `../` would point to the parent folder. As a second safeguard, the program refuses to write any output whose name is not a plain file name. The six original requests are accounted for exactly as before.

Exercise 13.4

In a copy of `letters.csv`, set L06's time zone to `EST`. Predict the result. Why should the program keep this value rather than translate it to `America/Toronto`?

Solution. L06 is held with `outside_teaching_time_zone`. The summary reports 3 generated (L01, L02, L04) and 3 held (L03, L05, L06), and `letter_L06.html` is not produced. The two zones are not interchangeable on that date. On September 25, 2026, Toronto observes daylight time, four hours behind UTC, while EST is five hours behind UTC, so 14:00 EST is 15:00 in Toronto. Translating would either move the meeting by an hour or assume that the writer meant Toronto time. Only the person who arranged the meeting can say which time was meant, so the request goes back to that person.

Exercise 13.5

Copy `letters.csv` and add a seventh data record by copying L06. Keep its identifier L06 and change only its recipient to `Example Recipient 7` and its matter to M01. What happens in a simple generator that uses identifiers as filenames? What does the supplied program do?

Solution. A simple generator would write `letter_L06.html` twice. The second letter, to Example Recipient 7 for M01, overwrites the first, so the M03 confirmation to Example Recipient 6 disappears without a trace, and the folder still looks complete, with one file per identifier. The supplied program holds both lines with `duplicate_letter_id` (source rows 7 and 8). The summary reports 7 input rows, 3 generated (L01, L02, L04), and 4 held (L03, L05, and both L06 lines), and no `letter_L06.html` is written. Someone must decide which request is genuine, or give one of them a new identifier, before either letter is drafted.

Chapter 14

Track Professionally Supplied Dates

Exercise 14.1

Name the six baseline classifications, and for each entry that needs review, give the issue that caused it.

Solution. As of September 24, 2026: D01 is `upcoming` (September 25); D02 is `today`; D03 is `past_recorded_date` (September 23); D04 and D05 are `needs_review`; D06 is `recorded_complete`. D04's issue is `invalid_supplied_date`: 2026-09-31 has the right form but does not exist, because September has thirty days. D05's issue is `missing_verifier`: its `verifier` cell is blank. The summary reports six input rows, two needing review, and one in each of `upcoming`, `today`, `past_recorded_date`, and `recorded_complete`; `superseded` is 0.

Exercise 14.2

Before running `--as-of 2026-09-25`, predict every classification and every summary count that will change; then run it and compare.

Solution. The verification dates, September 20, are still no later than the reporting date, so no new issue arises. D01 becomes `today`. D02 and D03 are both `past_recorded_date`. D04 and D05 still need review for the same reasons, and D06 is still `recorded_complete`. The summary shows `reporting_date` 2026-09-25, `past_recorded_date` 2 (was 1), `today` 1 (now D01 instead of D02), and `upcoming` 0 (was 1). `needs_review` stays 2, `recorded_complete` 1, and `input_rows` 6. No entry in `dates.csv` changed; the `run.json` parameters record the different reporting date.

Exercise 14.3

Copy `dates.csv` and mark D01 `superseded`. Add D07 for M01 and TASK01, with supplied date 2026-09-28, source DATE-SOURCE-07, verifier Reviewer Example, verification date 2026-09-22, owner A, completed `no`, status `current`, and `supersedes` D01. Predict the register. Then, in a fresh copy, add the same D07 but leave D01 unchanged. Why do the results differ?

Solution. In the first copy there is exactly one current entry for M01 and TASK01, and D07's link names a single superseded entry for the same matter and task. The register has seven rows. D01 is classified `superseded`, with its original date 2026-09-25 preserved; D07 is `upcoming`. The summary shows `input_rows` 7, `superseded` 1, `upcoming` 1 (D07), and `needs_review` 2 (D04, D05), with the other counts unchanged.

In the second copy, D01 and D07 are both current for the same matter and task. Both need review with `conflicting_current_dates`. D07 also has `invalid_supersession_link`, because the entry it names, D01, is not marked superseded. The summary shows `needs_review` 4 (D01, D04, D05, D07) and `upcoming` 0. Neither the later date nor the later row wins. A supersession is an explicit statement in the record that one entry replaces another; a second current entry is a conflict that a person must resolve.

Exercise 14.4

Copy `dates.csv`, set D05's `completed` value to `yes`, and predict its classification. In a fresh copy, set D06's value to `Yes`, with a capital letter, and predict its classification and the summary counts.

Solution. D05 still needs review with `missing_verifier`. The completion flag is consulted only after every completeness check has passed, so it cannot rescue an entry without a verifier. The summary is identical to the baseline: `needs_review` 2 and `recorded_complete` 1 (still D06).

In the second copy, `Yes` is not one of the two permitted values. D06 moves from `recorded_complete` to `needs_review` with `invalid_completion_flag`. The summary shows `needs_review` 3 (D04, D05, D06), `recorded_complete` 0, and `upcoming`, `today`, and `past_recorded_date` each 1, as before. The program does not guess that `Yes` means `yes`; a person corrects the cell in a copy.

Exercise 14.5

Write two or three checks that use named entries. Each must pass for the reference and fail for the date-first answer in *A Plausible Wrong Answer*.

Solution. Any of the following works.

1. The register has six rows, one for each input entry, and D04 is among them with `needs_review` and `invalid_supplied_date`. The date-first answer accounted for five entries and skipped D04.
2. D05 is `needs_review` with `missing_verifier`, not `upcoming`. The date-first answer listed D05 as upcoming.
3. D06 is `recorded_complete`, and the only `past_recorded_date` entry is D03. The date-first answer reported two passed dates, D03 and D06.

The reference's `date_register.csv` satisfies all three. Checks of this kind test identities, not just counts: a different wrong answer could produce the right number in one category with the wrong entries in it.

Chapter 15

Search Names and Parties

Exercise 15.1

Use `--exact-only` with the default query. Which candidates appear, and which columns show that they are separate entities?

Solution. The result is identical to the baseline: two candidates, N01 and N02, both `exact_primary_name` with matched text Alex Morgan and the explanation "normalized exact match." `matches` is 2. Only the `name` column agrees. The `entity_id` (N01, N02), `role` (former client, opposing party), `matter_id` (M01, M02), `source_id` (ENTITY01, ENTITY02), and `aliases` (A. Morgan, blank) all differ. The register supplies two entities; nothing in it says they are one person, and the program does not infer that they are.

Exercise 15.2

Try `A. Morgan` and `A Morgan` without the period, each with and without `--exact-only`, predicting each candidate's match type before you run. Why is N02 absent from every result?

Solution. `A. Morgan` returns one candidate in both runs: N01, `exact_alias`, matched text A. Morgan. `A Morgan` normalizes to `a morgan`, which differs from `a. morgan` by one deleted character, the period. With possible matches on, it returns one candidate: N01, `possible_alias`, matched text A. Morgan, explanation "one insertion, deletion, or substitution." With `--exact-only` it returns nothing (`matches` 0). N02 never appears because it has no alias, and its primary name, `alex morgan`, is more than one edit from either query. The program does not expand an initial to a first name; whether "A. Morgan" could refer to N02 as well is a question for a person.

Exercise 15.3

Search for `Morgan`. Why is the result empty? Describe it accurately in one sentence, without treating it as a clearance.

Solution. The result has no candidates (`matches` 0). The query `morgan` is compared with each whole normalized name, and `alex morgan` and `a. morgan` are five and three characters longer, far beyond one edit. The search does not look for a word inside a name. An accurate sentence: "A search of the party register with SHA-256 fingerprint 4f3a80cd... for the query 'Morgan', using normalized exact and one-edit matching against whole primary names and aliases, returned no candidates." The register in fact contains two entities with the surname Morgan, which is why the sentence must describe the search, not the world.

Exercise 15.4

Search for `MAPLE  SERVICES  LTD`, in capitals and with two spaces between words, and then for `Maple Services`. Predict the candidates, match types, and matched text for both queries.

Solution. After normalization the first query is `maple services ltd`, equal to N03's primary name. It returns one candidate: N03, `exact_primary_name`, matched text Maple Services Ltd. The second query, `maple services`, is four characters short of the primary name, so the primary name does not match. It is one inserted *s* away from the alias `maple service`, so it returns one candidate: N03, `possible_alias`, matched text Maple Service. With `--exact-only` the second query returns nothing. The program found N03 through a supplied alias and a spelling rule, not because it knows that `Ltd` is a corporate suffix.

Exercise 15.5

Copy the chapter folder and add N05: Jordan Example, prospective contact, matter M05, source ENTITY05, with no alias. Run the default search again. Which values in `summary.json` and `candidates.csv` change, and why does the changed value matter when you examine a search saved last month?

Solution. The candidates are unchanged: N01 and N02, both `exact_primary_name`, and `matches` is still 2. The query and scope lines are unchanged. Only `register_sha256` changes, because the file's bytes changed; it no longer begins `4f3a80cd`. That fingerprint is what ties a saved result to the register it searched. A search saved last month describes last month's register. If the fingerprint of today's register differs, the old result does not cover the new entries, and the search must be run again; identical candidate lists do not show that the registers are the same.

Chapter 16

Organize Legal Authorities

Exercise 16.1

Account for all eight retrieved records: name the records behind the five authorities, the six authority-versions, and the one ungrouped entry.

Solution. Q08 has no authority ID, so it is ungrouped with `missing_authority_id`; that leaves seven grouped records. Their authority IDs give five authorities: DEC01 (Q01, Q02), DEC02 (Q03), LEG01 (Q04, Q05), COM01 (Q06), and DEC03 (Q07). The authority-versions are DEC01/2026, DEC02/2026, LEG01/2024, LEG01/2026, COM01/2026, and DEC03/2026: six, because LEG01 appears in two versions. Seven grouped plus one ungrouped is eight, the number of input records.

Exercise 16.2

Why must DEC01 and DEC02 stay separate? Name the values in `membership.csv` that a test should check, and say what summary counts merging them would produce.

Solution. The register supplies DEC02 as a different authority: Q03's `relationship` is `separate later decision`. A later decision in the same proceeding is a distinct source that may say something different; treating it as a copy would make anything attributed to DEC01 appear to rest on a document it does not come from. A test should check that Q01 and Q02 have authority ID DEC01; that Q03 has authority ID DEC02, `related_authority` DEC01, and `relationship separate later decision`; and that the summary reports 5 authorities and 6 authority-versions. A merge, whether by grouping on the related authority or by Q03 carrying DEC01 as its authority ID, gives 4 authorities and 5 authority-versions, with DEC01 holding Q01, Q02, and Q03. The test must fail on those values.

Exercise 16.3

Q07 is grouped while its verification is pending. Copy `authorities.csv`, set its `verification` to `recorded`, and leave its reviewer empty. Predict the outputs. Why does that field value alone fail to establish verification?

Solution. Q07 is grouped because grouping requires only a record ID, authority ID, version, and source ID, and Q07 has all four. In the copy, nothing changes except the `verification` cell of Q07 in `membership.csv`, which now reads `recorded` beside a blank reviewer and review date. Q07 has no issue, and the summary is unchanged: 8 records, 5 authorities, 6 authority-versions, 1 ungrouped. The register displays verification status as supplied; it does not check it. (The worksheet, by contrast, reports `missing_reviewer` for a proposition marked `recorded` without a reviewer.) A cell reading `recorded` shows only that someone typed that word, here with no reviewer and no date beside it. It does not show that anyone has read the source, confirmed the citation, checked which version applies, or confirmed that it says what a draft relies on. A verification status needs an identifiable reviewer, a date, and the version reviewed, and even then it is a record of a person's review, not of the program's.

Exercise 16.4

Copy `propositions.csv`, change P02's version from 2024 to 2026, and predict its issues. In a fresh copy, change the version to 2025 and predict again. What does the first result leave unestablished?

Solution. With version 2026, P02 has no issue, because LEG01/2026 (Q05) is in the register. With version 2025, P02 has the issue `unknown_authority_version`, because the register has LEG01 only in 2024 and 2026. In both copies the register and its summary are unchanged. The first result says only that the version P02 names exists. It does not say that 2026 is the version applicable to the question P02 addresses; P02 was described as a historical-version note, and the change may have pointed it at the wrong edition. Choosing the applicable version is part of the research, not of the register.

Exercise 16.5

Copy `authorities.csv` and give Q08 the authority ID DEC01. Predict the four summary counts and the records in DEC01's group. Who made the decision to treat Q08 as a copy of DEC01?

Solution. Q08 now has every required field, so it is grouped. The summary shows 8 records, 5 authorities, 6 authority-versions, and 0 ungrouped: Q08's version, 2026, is the version DEC01 already has, so neither the authority count nor the version count changes. DEC01 now has Q01, Q02, and Q08, and `ungrouped.csv` has only its header. The person who typed DEC01 into the cell decided it. The program accepts the supplied identifier; it does not compare Q08 with Q01 or Q02 to see whether they are the same document. The change that matters is therefore visible only in the membership list and in the record of who changed the input, not in the authority counts.

Chapter 17

Compare Document Versions

Exercise 17.1

For P01–P04, list each changed word or number and give each paragraph's status.

Solution. P01 is `changed`: *may* is replaced by *must*, and *10* by *14*. P02 is `changed`: *not* is deleted, so "shall not disclose" becomes "shall disclose." P03 is `changed`: in the amount, the unit *10* is replaced by *100*, so $10,000 becomes $100,000. P04 is `unchanged`. No paragraph changed position. The summary reports 4 paragraphs, 3 changed, 1 unchanged, and 0 position changes.

Exercise 17.2

Explain why "no change in the extracted body text" says less than "no change in the complete source document".

Solution. A source document can contain much that body-text extraction may omit or alter: footnotes and endnotes, headers and footers, tables, text boxes, comments, tracked changes not yet accepted, defined-term formatting, schedules and attachments, embedded objects, and text in scanned images. Extraction can also reorder text or change characters such as dashes and quotation marks. A comparison of extracted text can only report on what the extraction produced. "No change" therefore

holds for that text under that comparison rule. Whether the complete documents are the same needs a check of the extraction itself, or a comparison of the files in their original form.

Exercise 17.3

Copy the chapter folder and move P04's line to the top of `revised.txt`, keeping all text unchanged. Predict every paragraph's status and `position_changed` value, then the summary counts.

Solution. Statuses are unchanged, because paragraphs are matched by identifier: P01, P02, P03 `changed`, P04 `unchanged`. All four paragraphs have `position_changed` 1. P04 moves from position 4 to 1, and P01, P02, and P03 each move down one place, from 1, 2, 3 to 2, 3, 4. The summary reports 4 paragraphs, 3 changed, 1 unchanged, and `position_changes` 4. One moved paragraph changes the position of every paragraph it passes; the report records the movement without inventing any change in wording, and the report page notes "Paragraph position changed" for each.

Exercise 17.4

Copy the chapter folder and, in P04 of `revised.txt`, put two spaces between *keep* and *a*. Predict the summary for an exact comparison, then for a run with `--whitespace`. In the second run, what revised text does `comparison.csv` show for P04? Why can the report still mark the whitespace difference although the status is `unchanged`?

Solution. In the exact comparison, P04 becomes `changed`: the summary reports 4 paragraphs, 4 changed, 0 unchanged, 0 position changes, and `comparison exact paragraph text`. With `--whitespace`, the extra space is ignored: 3 changed (P01, P02, P03), 1 unchanged (P04), and `comparison whitespace-normalized`. In both runs, `comparison.csv` shows P04's revised text exactly as it is in the file, with two spaces between *keep* and *a*. The normalization governs the status comparison, not what is preserved. The report builds its visible difference marks from the raw original and revised text, so it can mark the extra space even though the normalized status is `unchanged`. The summary records which comparison rule produced the result.

Exercise 17.5

Copy the chapter folder and add a fifth line to `revised.txt`: P05, a tab, then "The parties may sign separate copies." Predict P05's row in `comparison.csv` and the summary counts. Which paragraphs now contribute to the `changed` count?

Solution. P05's row has `original_position` 0 (absent from the original), `revised_position` 5, status `inserted`, `position_changed` 0, an empty original text, and the new sentence as revised text. P01–P04 are as before. The summary reports 5 paragraphs, 4 changed, 1 unchanged, and 0 position changes. `changed` counts every paragraph that is not `unchanged`: here three paragraphs whose wording changed and one inserted paragraph. To know which kind each is, read the `status` column; the count alone does not distinguish them.

Chapter 18

Check Time Entries and Totals

Exercise 18.1

Calculate the baseline total and all three matter subtotals by hand. List the entries in each subtotal.

Solution. T01 is $30 \times 240/60 = 120.00$, T02 is $45 \times 240/60 = 180.00$, T03 is $15 \times 300/60 = 75.00$, and T05 is $60 \times 200/60 = 200.00$; T04 and T06 are held and contribute nothing. M01 totals CAD 300.00 (T01 and T02), M02 CAD 75.00 (T03 only), and M03 CAD 200.00 (T05 only). The total is CAD 575.00, as in `summary.json` and `matter_totals.csv`.

Exercise 18.2

Copy the input file and add a seventh data record identical to T02. Predict the calculable total, the held count, and M01's subtotal. Explain why simply removing the second T02 would be the wrong repair.

Solution. The file now has seven rows. Both T02 rows, at source rows 3 and 8, are held with `duplicate_entry_id`, and T04 and T06 remain held: four held, three calculable (T01, T03, T05). The total is CAD 395.00, and M01's subtotal falls to CAD 120.00 (T01 only); M02 and M03 are unchanged at CAD 75.00 and CAD 200.00. Removing only the second T02 would restore CAD 575.00 and remove the duplicate warning, although T04 and T06 would still need review. It would let

file order decide which T02 row is genuine. The two rows may be an accidental copy or two separate pieces of work under one wrongly repeated identifier; only the source records can say which, so both wait.

Exercise 18.3

Copy the input file and enter `0` for T04's minutes. Predict the summary, and explain why the original blank cannot be treated as that zero.

Solution. T04 becomes calculable with amount 0.00. The summary reports six input rows, five calculable, one held (T06), and a total of CAD 575.00, the same total as the baseline. A blank says the duration is unknown; a zero says someone recorded a duration of nothing. Reading the blank as zero would assert a fact nobody supplied, and it would remove T04 from the held list, so no one would ask how long the work took. Because the total cannot show the difference, only the calculable count and T04's ledger line reveal it.

Exercise 18.4

Copy the input file and change T05's currency to `USD`. Predict its ledger line, the total, and `matter_totals.csv`. Explain what you can and cannot conclude when M03 is absent from that file.

Solution. T05 is held with the reason `wrong_currency` and an empty amount. Three entries are calculable (T01, T02, T03) and three held (T04, T05, T06); the total is CAD 375.00. `matter_totals.csv` lists only M01 300.00 and M02 75.00. M03 is absent because neither of its entries is calculable: T05 is in another currency and T06 has negative minutes. The absence does not mean that M03 has no recorded time or that its time is worth nothing; the ledger still shows both M03 entries. The program also does not convert USD to CAD, which would need a rate and a date that nobody supplied.

Exercise 18.5

Design a two-row input that gives three different totals under these methods: the reference rule; half-even rounding on each line; and rounding only after adding the unrounded amounts. Give all three totals.

Solution. Use two rows, each one minute at CAD 240.30 an hour, for example `X1` and `X2` for M01 in CAD with their own source references. Each line is CAD 4.005 before rounding. (a) The reference rounds each line half up to 4.01 and sums them: CAD 8.02. (b) Rounding half to even on each line gives 4.00 twice: CAD 8.00. (c) Rounding the unrounded sum once gives 8.010, which is CAD 8.01. One input, three

answers, so a single test that expects 8.02 rejects both alternatives. The release's `test_54_time_total_of_rounded_lines` uses exactly this input and expects 8.02. (Added to the baseline file as T07 and T08, the same two lines give a worksheet total of CAD 583.02.)

Chapter 19

Review Files Recorded as Closed

Exercise 19.1

Account for all five register entries by category. For each entry, name the step that determines its category.

Solution. Step 1 decides F03: its hold flag is `yes`, so it is `hold`. Step 2 decides F04 (`needs_review`, `invalid_review_date`: the review date is blank) and F05 (`needs_review`, `invalid_closure_date`: the closure date is blank). Step 3 decides F01, due for review because September 20 is on or before September 24, and F02, upcoming because October 1 is later. One due, one upcoming, one hold, two needing review: five entries, five classifications.

Exercise 19.2

In a copy, give F01 a September 24 review date; it is complete and has no hold. Predict its category in a September 24 report, then test it.

Solution. It is due for review, because the rule includes a review date equal to the reporting date. In a copy with F01's review date set to `2026-09-24`, F01 is still `due_for_review`, and the summary is unchanged: due 1, upcoming 1, hold 1, needs review 2. The release's `test_58_closing_today_and_history` checks this case. Making the boundary exclusive would be a new rule, needing new tests.

Exercise 19.3

Copy the input and remove F03's review date. Predict its category, its `issues`, and the summary. Explain which information must remain visible and why.

Solution. F03 stays `hold`, and its `issues` cell now reads `invalid_review_date`. The summary does not change: due 1, upcoming 1, hold 1, needs review 2. Two things must survive. The hold must stay the classification, because step 1 comes before the completeness check; an incomplete record must never drop out of the hold category, or into a list that someone might act on. And the missing date must be recorded, so that someone supplies it. The release's `test_57_closing_hold_does_not_disappear` checks both.

Exercise 19.4

Predict every category for reporting date 2026-07-31. Then run the supplied program on the unchanged files with `--as-of 2026-07-31`. Explain F03's result.

Solution. Every supplied closure date in the register is August 1, 2026, which is later than the reporting date, and F05 has none, so every entry fails the closure-date check. F01 and F02 become `needs_review` with `invalid_closure_date`; F04 has `invalid_closure_date;invalid_review_date`; F05 keeps `invalid_closure_date`. F03 remains `hold`, now with `invalid_closure_date` in its `issues`, because step 1 comes first. The summary: reporting date 2026-07-31, hold 1, needs review 4, due 0, upcoming 0. As of July 31, the register claims closures that had not yet happened, so it cannot be used to classify anything for that date.

Exercise 19.5

Copy the input and change F03's hold flag from `yes` to `Yes`. Predict its output line and the summary, and explain why the program takes this cautious approach. What must someone reading the summary notice?

Solution. F03's line keeps `Yes` in the `hold` column, with classification `needs_review` and issue `unknown_hold_flag`. The summary reports hold 0, needs review 3, due 1, upcoming 1. The program does not guess that `Yes` means `yes`; but neither does it read an unrecognized flag as `no`, which would make F03, with its September 1 review date, due for review. Sending it to a person is the only choice that cannot put a held file on the due list. A reader of the summary must notice that the hold count fell from one to zero: the hold has not been lifted, it has moved to `needs_review`, and the `issues` column says why. The repair is a corrected source value, not a more lenient program.

Chapter 20

Test the Applications

Exercise 20.1

The test `test_24_large_selected_identities` requires four selected events for M0001. Describe a wrong selection that a count-only check would still pass. What assertion would reject it?

Solution. Every large matter has four events. A program that selected the wrong matter, for example M0002's events E00005–E00008, or any four events at all, would satisfy a count of four. The test therefore also reads the selected `accepted.csv` and asserts that its event identifiers are exactly E00001, E00002, E00003, E00004, in that order. That assertion fails for any other four.

Exercise 20.2

The test `test_07_bad_request_holds_dependent_response` removes E01's source reference from the clean event file. Give the accepted and held counts, and the reason for holding each held record. Why would checks of each record's own fields miss one of the problems?

Solution. Sixteen events are accepted and two held. E01 is held for its blank source, with the reason `missing_source_id`. E02 is held with `request_needs_review`: it is the response to request R01, which E01 records, and a response cannot be placed on a request that is itself held. Every one of E02's own fields is valid: its date, 2026-09-04, follows the request; its matter and client agree with the register; its source is SRC02. A check that looked at each record's cells separately would accept it. Only a check of the relationship between E02 and E01 holds it, and the test asserts exactly the pair {E01, E02} and E02's reason.

Exercise 20.3

What does the endpoint mutation establish, and what remains unchecked? Consider the matcher's other inclusive comparison, the one that decides whether a request's window is still open. You can answer from the written rule and saved failure log; rerunning the mutation is optional.

Solution. It establishes that this particular defect, excluding day 14 in the comparison that decides each response's eligibility, is detected: in the mutated copy, `test_15_matching_day14_and_day15` and `test_20_matching_duplicate_response_id` fail on the rule, and `test_chapter_20` fails because the helper can no longer find the text it mutates. It does not establish that other defects are detected. The matcher's other inclusive comparison, which decides whether a request without a response is still `window_open`, is untouched by this mutation, and `test_15_matching_day14_and_day15` says nothing about it. Mutating that comparison separately (from `<=` to `<`) is caught by a different set of five failures: `test_chapter_23`, the Chapter 23 part of `test_01_all_hand_derived_baselines`, `test_63_worklist_baseline_values`, `test_66_worklist_future_response`, and `test_mixed_worklist_named_expectations` in `test_revision_cases.py`, which checks the nine-request mixed case by identity. Each comparison needs its own challenge, and the mixed case catches it because Q05 sits exactly on day 14. Neither mutation shows that the rule is the right one for an office, or that the suite catches every defect.

Exercise 20.4

Work on a copy of the release folder, either locally or through an assistant that can edit and execute uploaded programs. In `programs/legal_apps.py`, change the time-entry calculation from `rounding=ROUND_HALF_UP` to `rounding=ROUND_HALF_EVEN`, and add `ROUND_HALF_EVEN` to the `from decimal import` line. Predict which tests fail and which baseline test still passes. What `rounding` label does the faulty program print?

Solution. Three tests fail: `test_53_time_half_up_boundary` (`'4.00' != '4.01'`), `test_54_time_total_of_rounded_lines` (`'8.00' != '8.02'`), and `test_chapter_18` (the experiment's total is 579.00, not 579.01). The other 154 pass, including `test_01_all_hand_derived_baselines`: every baseline amount in Chapter 18 comes out in whole cents, so half up and half even agree on it. `test_55_time_duplicate` also passes, with CAD 395.00. The mutated program still prints `ROUND_HALF_UP per line; total is sum of rounded lines`, because that label is written into the program as text, not derived from the arithmetic. Only the half-cent tests can tell the difference.

Exercise 20.5

Run this locally from the release folder, or ask an assistant with the complete release to run it there:
`python3 -m unittest discover -p "test_*.py"`
Do not enter `cd programs` or add `-s programs`. What does the command print, and why does "no failures" alone fail to establish that the suite passed?

Solution. With Python 3.12 or later, it prints `Ran 0 tests in 0.000s` and then `NO TESTS RAN`, with exit status 5. Discovery starts in the current folder unless `-s` names another; the test files are in `programs`, which discovery does not enter from the release folder on its own, so no test was found. Python versions before 3.12 print `OK` after `Ran 0 tests`. A run in which nothing could fail is not evidence that anything passed. Read the count every time: this release must report 157.

Chapter 21

Package a Reproducible Demonstration

Exercise 21.1

Group the demonstration archive's twelve members by role. Which member is absent from the manifest, and why?

Solution. Five program files: the shared `matter_tools.py`, the three entry points `matter_view.py`, `matter_check.py`, and `matter_run.py`, and the test file `test_matter_tools.py`. Five data files: `matters_demo.csv`, `matter_events_demo.csv`, `matter_events_problem.csv`, `matters_large.csv`, and `matter_events_large.csv`. One instruction file, `README_DEMO.txt`. These eleven are the allowlist and the manifest's eleven entries. The twelfth member, `MANIFEST.json`, is added by the maker and does not list itself: writing its own fingerprint into it would change its bytes, and so its fingerprint.

Exercise 21.2

Extract the archive into a new empty folder, delete `programs/matter_tools.py` there, and run the tests. What happens? Why might tests run in the release folder miss a package that omitted this file?

Solution. The test file stops at once with `ModuleNotFoundError: No module named 'matter_tools'`, because its import of the shared module fails. In the release folder the same command would pass, because `programs/matter_tools.py` is there whatever the archive contains; a package that forgot it would still appear to work for its author. The fresh folder contains only what the package supplied, so it tests the package rather than the author's computer.

Exercise 21.3

Three manifest entries have the same fingerprint. Is that a problem? What does a matching fingerprint tell you about a received file, and what does it leave unanswered?

Solution. No. The three entry points contain identical bytes and call `main` in `matter_tools.py`; the three names match the three opening chapters. Compare their entries in the archive's `MANIFEST.json`. The three byte counts agree, and so do the three SHA-256 values. Identical bytes have identical fingerprints, whatever the files are called. A matching fingerprint establishes that the bytes you received are the bytes the manifest describes: the file was not altered or damaged after the manifest was made. It leaves open who made the manifest, since an altered package can carry an altered manifest that matches it; whether the code is correct; whether the data are true; and whether the package is suitable or approved for anyone's use. Knowing that the manifest itself is genuine needs separate evidence, such as its fingerprint obtained directly from the author by another route.

Exercise 21.4

Run the package maker locally, or ask an assistant that can execute it to make two archives. Build the demonstration twice under different names, at least three seconds apart. Compare the SHA-256 fingerprints of the two ZIP files, then of their `MANIFEST.json` members. Explain what you find. In what sense is this package reproducible?

Solution. The two archive fingerprints differ. The two `MANIFEST.json` members have the same fingerprint, and it is also the fingerprint of the manifest inside the saved archive `reference_outputs/experiments/ch21/Matter_Demo.zip`; compute it with `python3 -m zipfile -e` followed by a SHA-256 tool, or compare the extracted manifests byte for byte. (The value itself is not printed here: it changes

whenever any of the eleven payload files changes, as it did between releases.) A ZIP file records the date and time each member was written, so building at a different moment produces different archive bytes even when every member is identical. (Two builds within the same two-second interval can produce identical archives, because the ZIP format stores times to the nearest two seconds.) The package is reproducible in its contents: the same members, in the same order, with the same bytes, as the manifest shows. It is not reproducible as a single archive file. Compare packages by their manifests and member fingerprints, not by the fingerprint of the ZIP.

Exercise 21.5

Run the maker with `--root Chapters/chapter01` and a new output name. Predict its message and whether it creates an archive. Why is this better than including whatever matching files it can find?

Solution. It prints

```
Missing or symlinked required file: programs/matter_tools.py
```

It exits with status 2 and creates no archive. `Chapters/chapter01` holds the opening files side by side, with no `programs` or `data` folders, and has no test file, large files, or README, so the first listed path is not found. The maker checks the whole list before it writes anything. Packaging whatever it could find would produce an archive that looks complete, and the colleague would discover the missing pieces only when a command failed on her computer.

Chapter 22

Check Application-Review Records

Exercise 22.1

Which three reviews are complete on September 24? Give the issue code for each of the other two and say what action it calls for.

Solution. Specification (`REVIEW-1`, row 2), tests (`REVIEW-3`, row 4), and user guide (`REVIEW-5`, row 6) are `review_record_complete`. The user guide expires on September 24 and is still current on that day, because expiry is inclusive.

The program record (`REVIEW-2`, row 3) has `artifact_version_mismatch`: it reviews version 1, and the delivery is version 2. The action is a review of the program as delivered, recorded in a corrected copy of the register; adding a second program row instead would create a duplicate category (Exercise 22.4). The data-handling record (`REVIEW-4`, row 5) has `missing_reviewer`. The action is to consult the review record `REVIEW-4` and find out who performed the review, if anyone did; typing a name in order to pass the check would defeat it. The summary reports complete 3 and needs attention 2.

Exercise 22.2

Leave the register unchanged and run with `--as-of 2026-09-25`. Which record changes? Give its issue code and name the records that remain complete.

Solution. The user-guide record expired at the end of September 24. On September 25 it becomes `needs_attention` with `invalid_or_outdated_dates`. Specification and tests remain complete; their reviews expire on December 31. The program and data-handling records keep their issues, `artifact_version_mismatch` and `missing_reviewer`. The summary shows complete 2, needs attention 3, and reporting date 2026-09-25. The register file itself is unchanged; only the question's date moved.

Exercise 22.3

Leave the register unchanged and run with `--rule 2`, as though the rules had changed. Predict the summary and each row's issues. Which record should a reviewer examine first, and why?

Solution. No record is complete: the summary reports `complete` 0, `needs_attention` 5, with `rule_version` 2. Every row carries `rule_version_mismatch`, because all five reviews were of rule version 1. The program row also has `artifact_version_mismatch`, and the data-handling row also has `missing_reviewer`. A reviewer would start with the specification row: a change of rules is a change of specification, and the reviews of the program, the tests, and the guide depend on what the specification now says. The exercise shows that a review is tied to the exact rules reviewed, not only to the program file.

Exercise 22.4

Copy `reviews.csv` and add a sixth row: `program,2,1,Reviewer Example, 2026-09-20,2026-12-31,REVIEW-6`. Predict both program rows' statuses and issues, and the summary counts. Why does the newer-looking entry fail to make the program category complete?

Solution. The register has six rows. The original program row (row 3, `REVIEW-2`) is `needs_attention` with `duplicate_category;artifact_version_mismatch`. The new row (row 7, `REVIEW-6`) is `needs_attention` with `duplicate_category`, although it is otherwise valid. Specification, tests, and user guide stay complete; data handling still has `missing_reviewer`. The summary shows complete 3 and needs attention 3.

The rule requires exactly one record per category, and the program cannot know which of two records is authoritative: the second might be a correction, or a review of a different scope. A program that stored rows in a dictionary keyed by category would keep the last row, report four complete records, and silently lose `REVIEW-2`. Flagging both leaves the decision to a person, who can then prepare a corrected copy with one program row.

Exercise 22.5

Run the unchanged register with `--artifact 3`, as though version 3 were being delivered. Predict the complete and needs-attention counts and the data-handling issues. Then consider `--artifact 1`: which record, if any, is complete?

Solution. With `--artifact 3`, every row names version 1 or 2, so all five have `artifact_version_mismatch`: complete 0, needs attention 5, and the summary reports artifact version 3. The data-handling row carries two issues, `missing_reviewer;artifact_version_mismatch`; supplying its reviewer would no longer be enough. Reviews of version 2 do not transfer to version 3.

With `--artifact 1`, the program record is the only complete one: complete 1, needs attention 4. The other four rows name version 2 and mismatch, and data handling again has both issues. The same register gives three different answers for three declared deliveries, which is why the declared version belongs in the summary.

Chapter 23

Build an Integrated Matter Worklist

Exercise 23.1

Give each of the six baseline requests its status and three evidence fields. Why must Q06 retain its response rather than be labelled unanswered?

Solution. As of September 24, 2026, with the window 0–14 days (fields in the order `any_response`, `in_window`, `earliest`):

- Q01: `response_recorded`; yes, yes, unique; selected response CS01 (2 days).
- Q02: `window_open`; no, no, none (4 days old).
- Q03: `window_passed`; no, no, none (23 days old).
- Q04: `needs_review` with `invalid_request_date`; unavailable, unavailable, unavailable.
- Q05: `window_open`; no, no, none (exactly 14 days old, the last day of the inclusive window).
- Q06: `response_recorded`; yes, no, none; no selected response.

Q06 has a valid response, CS06, dated September 18 and observed before the reporting date. Its candidate row shows `elapsed_days` 17, `valid` 1, `observed` 1, `eligible` 0, and `outside_window`. The response is late, not absent. "No response" would misdescribe the record and could prompt someone to chase a response the office already holds.

Exercise 23.2

Predict the complete worklist for `--as-of 2026-09-25` before running the supplied program. Which request changes status? Which unanswered request stays open?

Solution. Q05 is now 15 days old, so it moves from `window_open` to `window_passed`. Q02 is only five days old and stays `window_open`. Q01 and Q06 remain `response_recorded`, Q03 `window_passed`, and Q04 `needs_review`. The summary shows response recorded 2, window open 1, window passed 2, needs review 1, not yet requested 0, and `in_window_yes` 1. The input files are unchanged; only the reporting date moved.

Exercise 23.3

Copy `responses.csv` and add `CS02,M01,Q01,2026-09-12,CR02`. Give Q01's status, three evidence fields, and selected response. Give CS01's and CS02's selection values in the candidate ledger. What changes in the summary?

Solution. Q01 remains `response_recorded`, with `any_response` yes, `in_window` yes, and `earliest ambiguous`. Its `selected_response` is blank: two responses share the earliest date, and the program does not choose between them. In `candidates.csv`, CS01 (row 2) and CS02 (row 4) are both valid, observed, eligible, and marked `tied_earliest`, each 2 days after the request; CS06 is unchanged. In the summary, `ambiguous_earliest` rises from 0 to 1. Everything else is unchanged, including `in_window_yes` 1 and all status counts: the tie affects which response is selected, not whether one exists.

Exercise 23.4

Run the unchanged files with `--matter M02`. Predict the rows in `worklist.csv` and `candidates.csv`, the summary counts, and `internal_validation.json`. Which files should go to a recipient for matter M02?

Solution. `worklist.csv` has two rows: Q03 (`window_passed`; no, no, none) and Q04 (`needs_review`, `invalid_request_date`; unavailable throughout). `candidates.csv` has only its header, because no response names M02. The summary shows requests 2, response recorded 0, window open 0, window passed 1, needs review 1, not yet requested 0, `in_window_yes` 0, and `ambiguous_earliest` 0. `internal_validation.json` still describes the whole input: `whole_requests` 6, with response recorded 2, window open 2, window passed 1, and needs review 1.

The worklist, candidate ledger, summary, and report contain only M02 and can form the deliverable, subject to the office's decision about recipients. `internal_validation.json` and `run.json`, which fingerprints the complete input files, are internal evidence that the whole file was checked before selection; they should not be sent automatically with the M02 report.

Exercise 23.5

Copy `assignments_large.csv`, remove M0002's record, and run the conversion. What does it print? Which files does it create? Why is this better than giving M0002's request a blank owner?

Solution. The copy has 2,999 assignments for 3,000 registered matters. The converter prints `Assignments must supply exactly one nonblank owner for every registered matter` and exits with status 2. It creates nothing: the output folder does not exist after the run. The check is made before any file is written, so there is no partial conversion to mistake for a complete one.

Converting anyway would produce request R0002 with no owner. An owner filter would then show it under neither A nor B, so each owner's list would look complete while one request belonged to nobody. Filling in an owner would be worse: it would invent an assignment decision. The remedy is to obtain the missing assignment from whoever is responsible for it, correct a copy of the table, and convert again to a new folder.